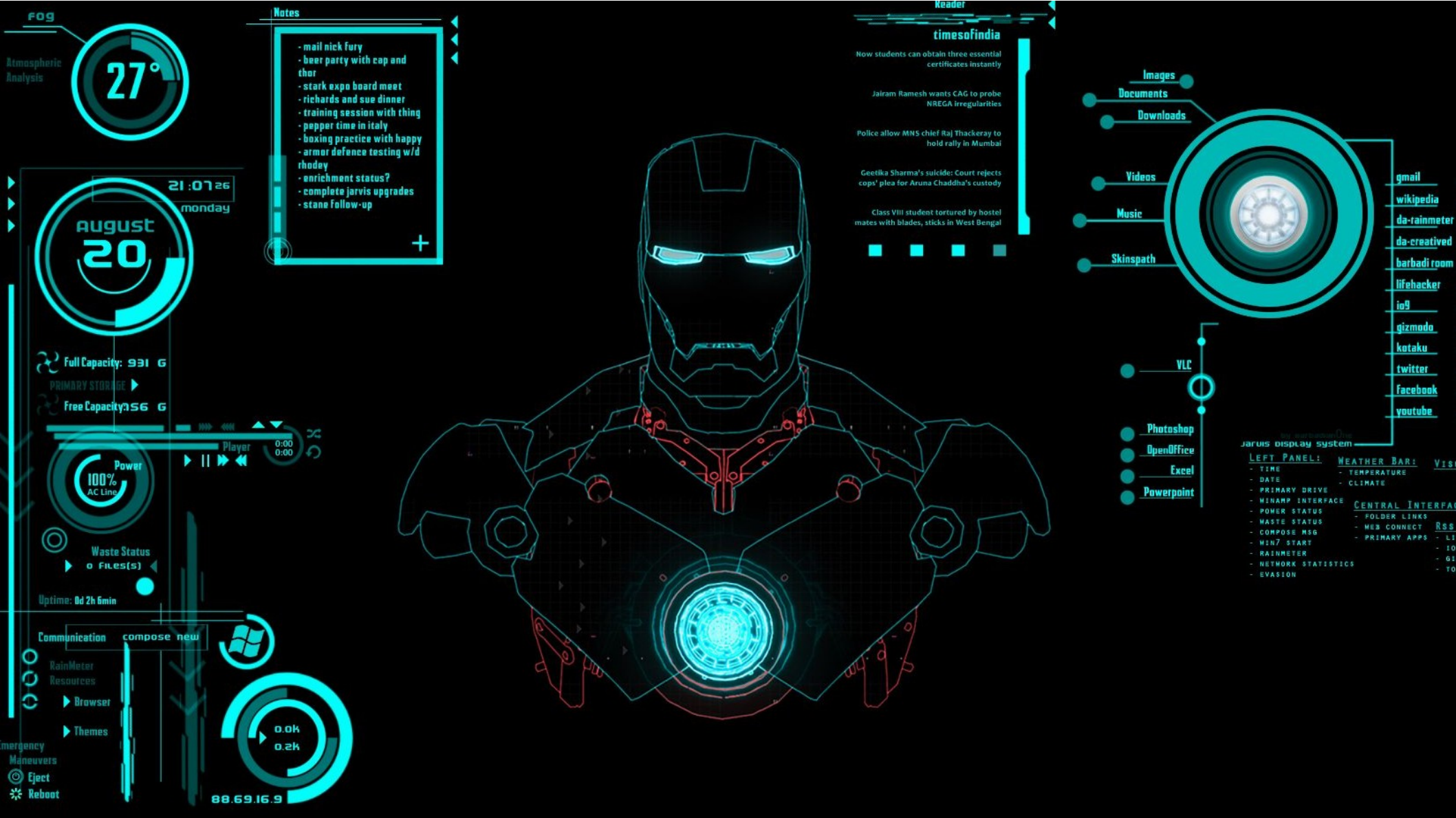




04. Processo de Visualização 2D

Prof. Alexandre Krohn

Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Roteiro

- **Processo de Visualização 2d (Bidimensional)**
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Processo de Visualização 2D (Bidimensional)

- Um processo de visualização converte um modelo de objetos em uma imagem, através de primitivas gráficas



Modelo X Imagem

- **Modelo** : Uma representação computacional (abstração) de um objeto, contendo as informações relevantes para uma dada aplicação
- **Imagem** : Uma representação gráfica, formada por pontos, sobre uma superfície

Modelo X Imagem

- Pode-se construir um sistema gráfico que manipula apenas imagens, sem manter modelo dos objetos que estão sendo criados.
 - Mas, nesse caso, após o desenho de uma entidade esta torna-se **apenas um conjunto de pontos** sobre a superfície de exibição(tela).
 - A partir de então, não é mais possível saber, dado um determinado ponto na tela, a que entidade este pertence, ou alterá-la para, por exemplo, reduzir seu tamanho : **Perde-se a referência do modelo**

Desenhando em 2D

- Nas duas primeiras aulas, estudamos algoritmos clássicos para desenho em 2D, na forma de primitivas : pontos, linhas, curvas, círculos
- Não nos preocupamos com os objetos em si

Propriedades dos Modelos

- Quando se trabalha com modelos, pode-se tratar as entidades levando-se em conta seu tipo, e com isto, é possível :
 - Determinar se um dado ponto pertence a entidade ou não,
 - Alterar propriedades como :
 - Posição
 - Tamanho
 - Orientação.

Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Etapa do Processo de Visualização 2D

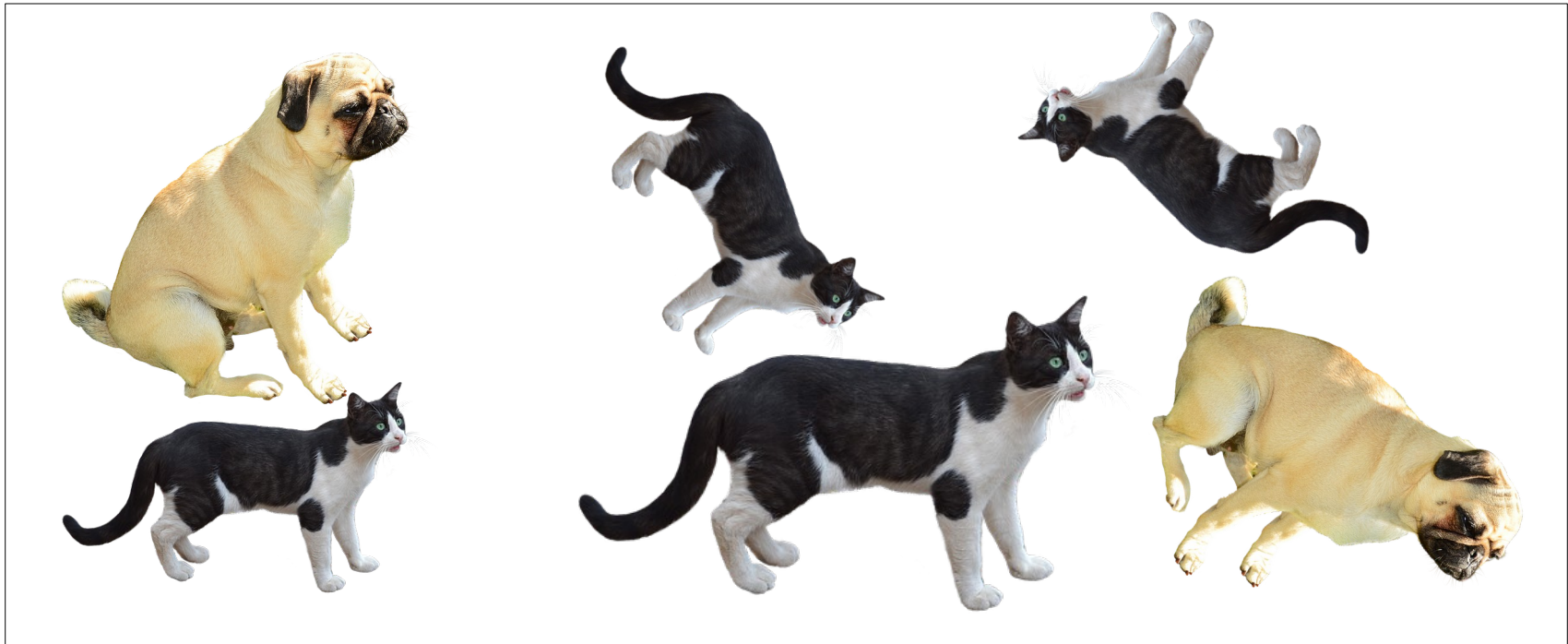
- A visualização 2D divide-se em três etapas, nessa ordem:
 - 1) Instanciação
 - 2) Recorte
 - 3) Mapeamento

Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Instanciação

- Na etapa de instanciação cria-se cópias a partir de um modelo (**POO**, vocês lembram?)
- Cada cópia, possui seus próprios valores e é chamada de **instância**:



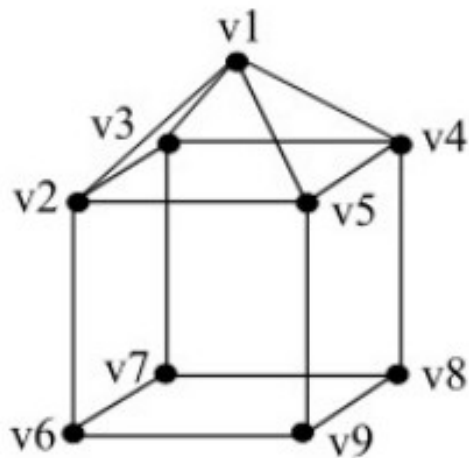
Instanciação

- Para armazenar as instâncias da aplicação, pode-se utilizar uma tabela com a classe do modelo e seus parâmetros de instanciação

Modelo	Posição X	Posição Y	Rotação	Escala
Cachorro	...	...	...	...
Gato	...	...	...	...
Gato	...	...	...	...
Gato	...	...	...	...
Gato	...	...	...	...
Cachorro	...	...	...	...

Modelos

- Internamente aos modelos, pode-se ter uma lista das primitivas que o compõe, e como elas estão ligadas
- Exemplo: **Malha de polígonos**



Vértices (geometria)

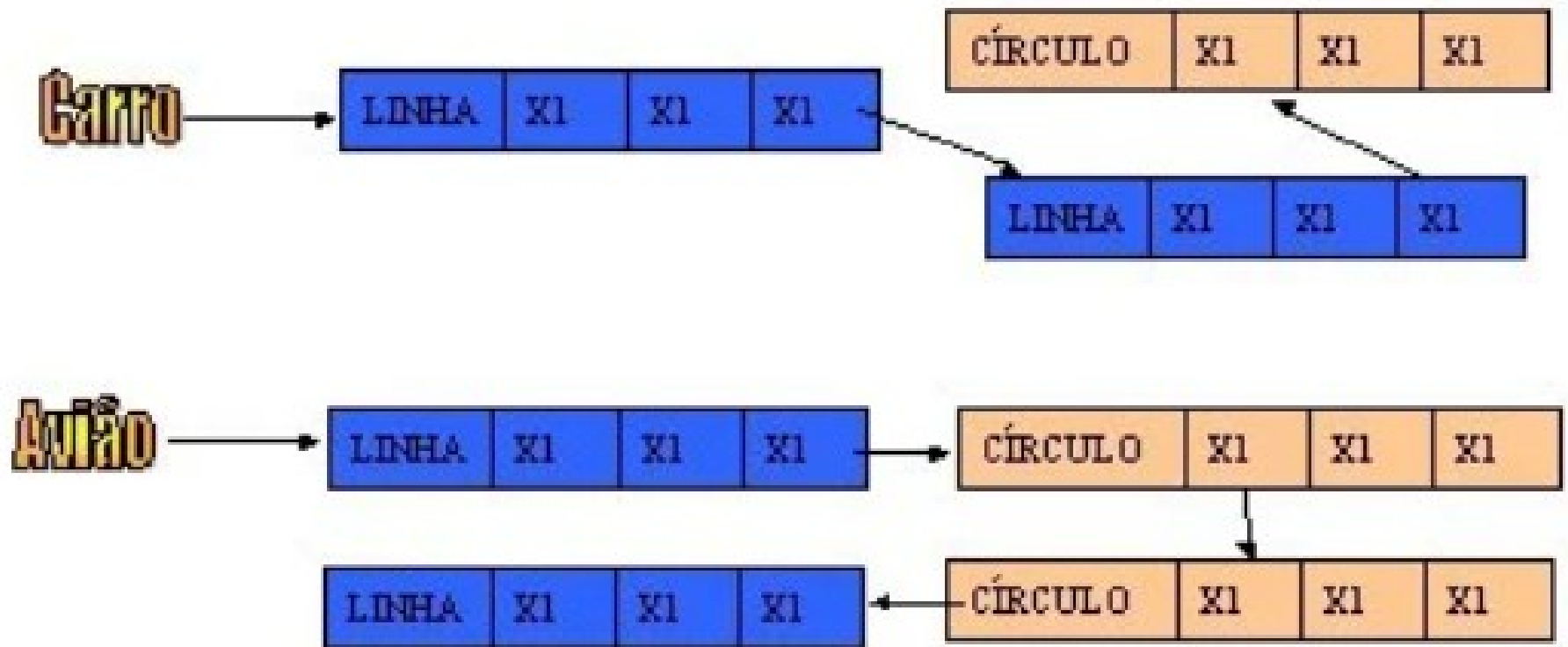
1	x1	y1	z1
2	x2	y2	z2
3	x3	y3	z3
4	x4	y4	z4
5	x5	y5	z5
6	x6	y6	z6
7	x7	y7	z7
8	x8	y8	z8
9	x9	y9	z9

Faces (topologia)

1	v1	v4	v5	
2	v1	v5	v2	
3	v1	v2	v3	
4	v1	v3	v4	
5	v4	v3	v7	v8
6	v5	v4	v8	v9
7	v2	v5	v9	v6
8	v3	v2	v6	v7
9	v6	v9	v8	v7

Modelos : Listas

- Pode-se também representar modelos por listas de primitivas

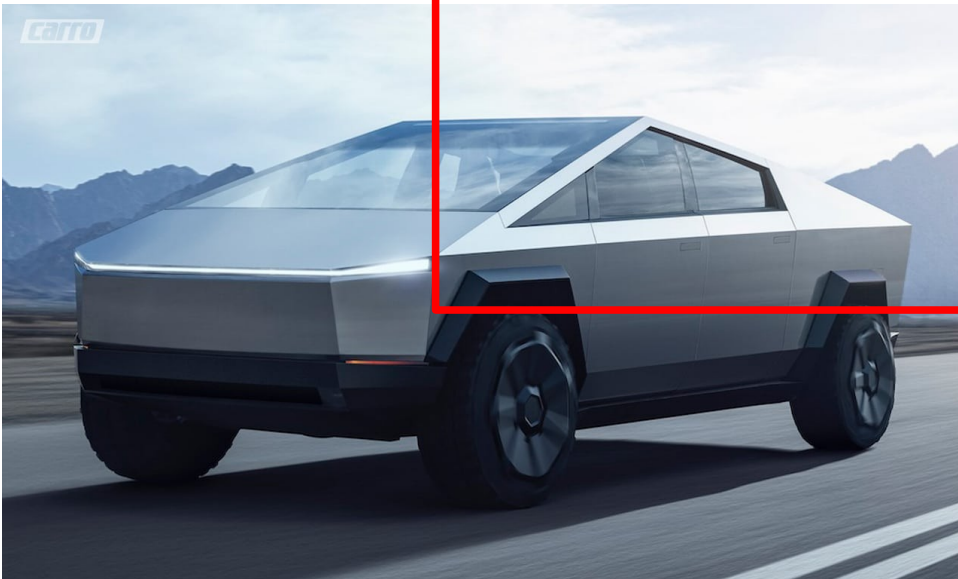


Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - **Recorte**
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Recorte

- Recorte, ou Clipping, é o processo de descarte de partes de um objeto a ser desenhado que ficam fora da “janela de visualização”

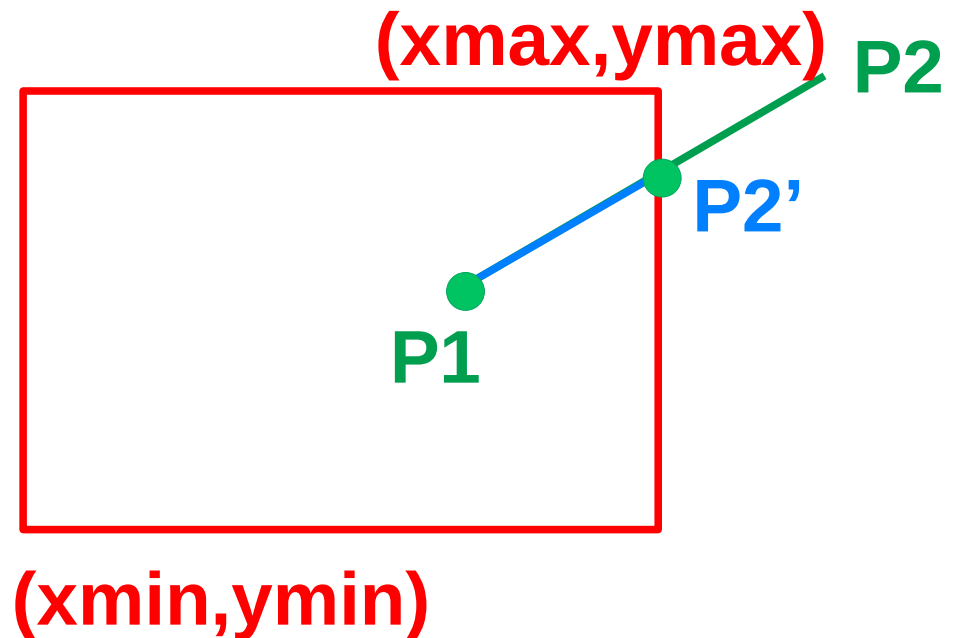
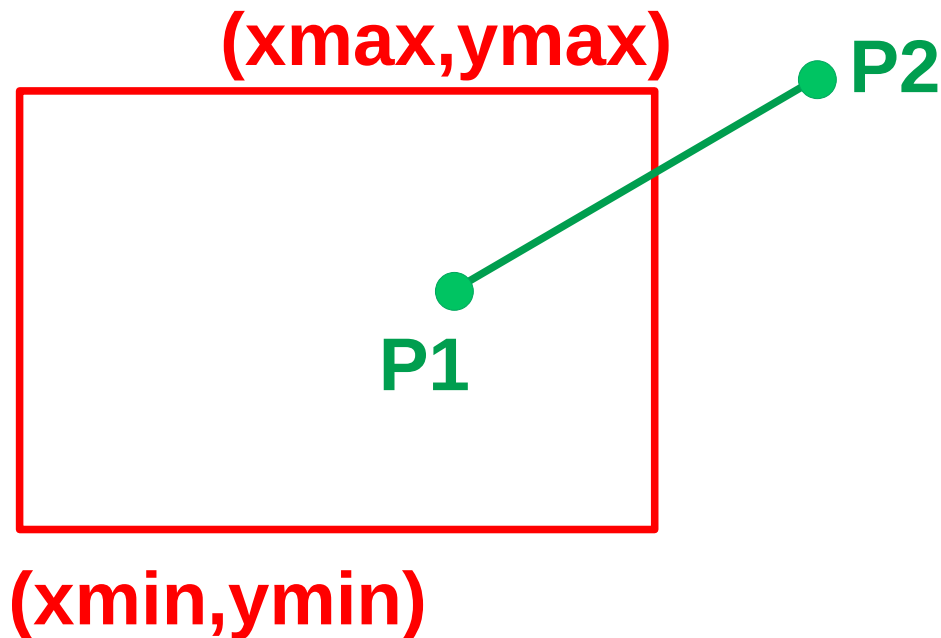


Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

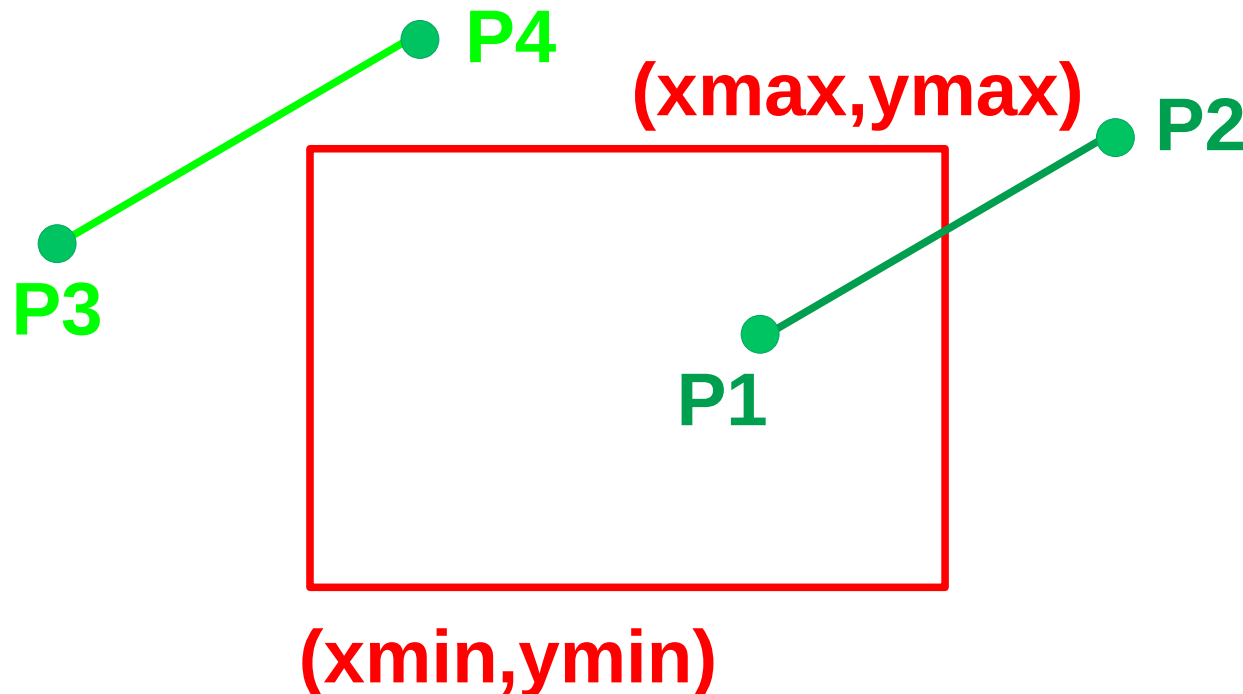
Recorte de retas

- Partindo-se de um segmento de reta $P1-P2$ e de um retângulo de visão $(xmin, ymin)$ e $(xmax, ymax)$



Recorte de retas

- Obtém-se um novo segmento de reta (*Pode ser nulo*)



Algoritmos Clássicos

- Para resolver esse problema, há dois algoritmos clássicos:
 - Cohen-Sutherland
 - Liang-Barsky
- E um mais moderno:
 - Matthes - Drakopoulos

Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Cohen-Sutherland

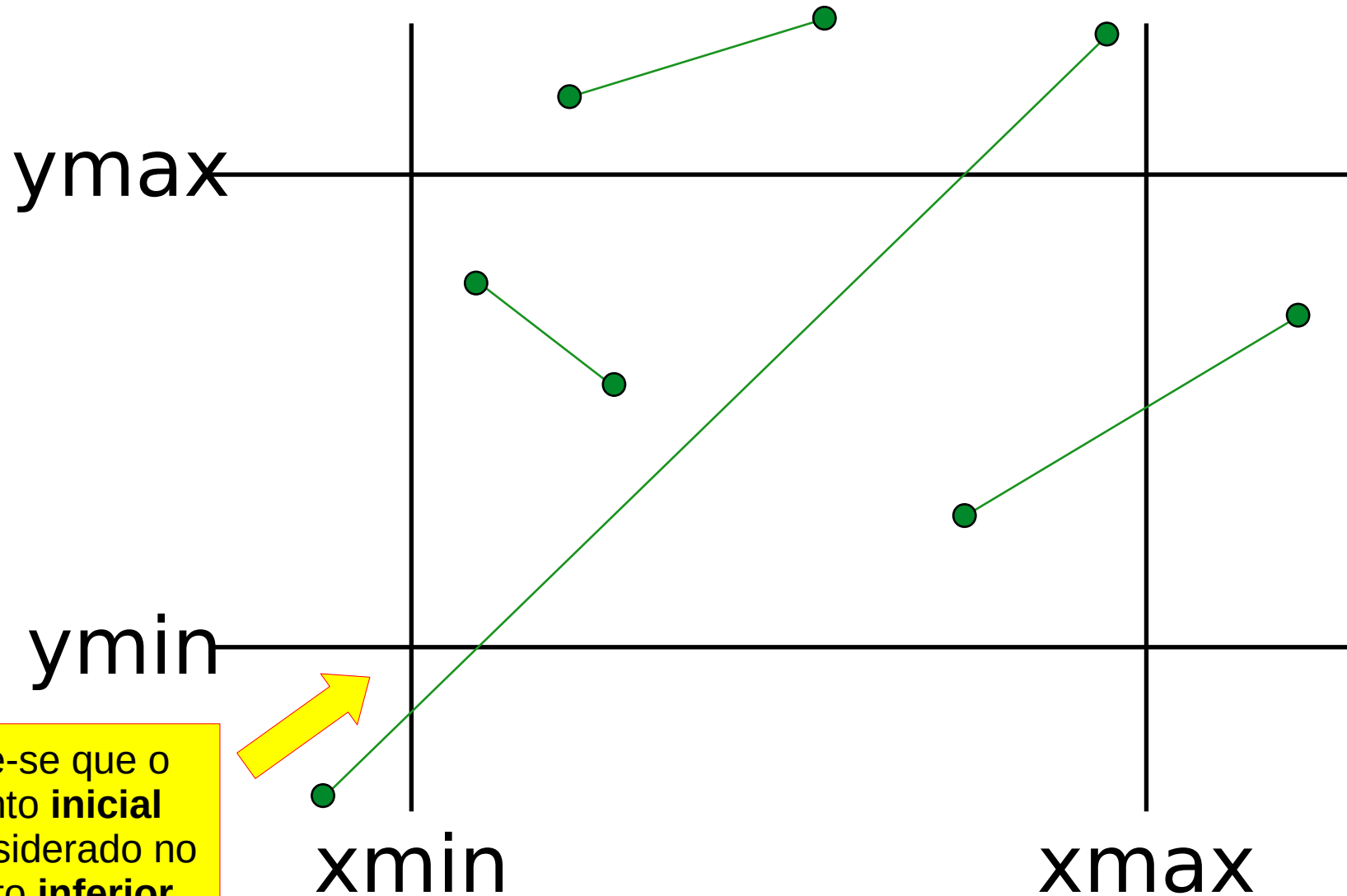
- No algoritmo de **Cohen-Sutherland**, os vértices do segmento são classificados com relação a cada semi-espaco plano que delimita o retângulo.

$$x \geq x_{\min} \text{ e } x \leq x_{\max} \text{ e } y \geq y_{\min} \text{ e } y \leq y_{\max}$$

Cohen-Sutherland

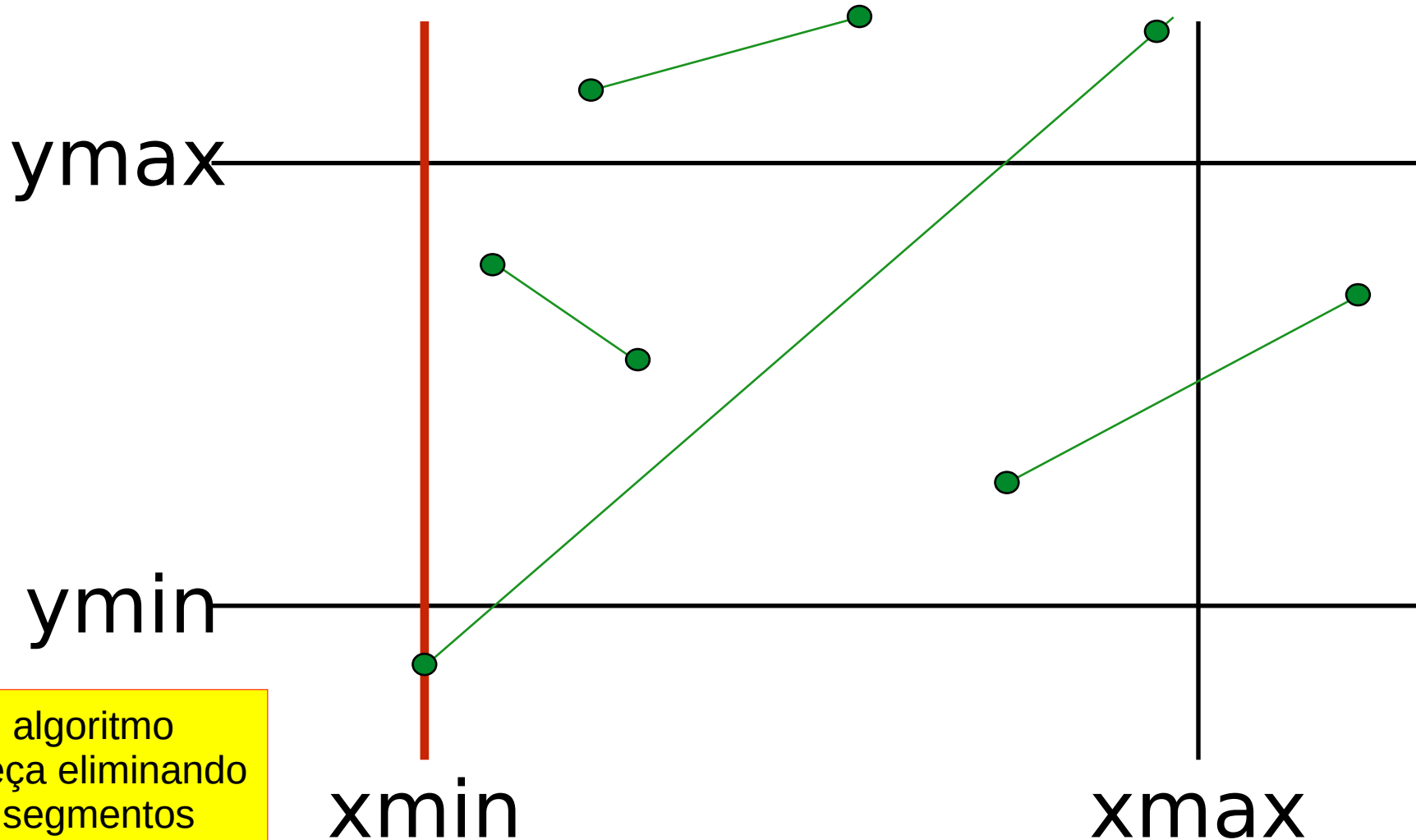
- Se **ambos** os vértices são classificados como “**fora**” em relação a um semi-espço, **descartar** o segmento.
- Se **ambos** são classificados como “**dentro**”: Testar o próximo semi-espço.
- Se um vértice está “**dentro**” e outro “**fora**”:
 - 1) calcular o ponto de interseção Q
 - 2) continuar o algoritmo com o segmento recortado

Cohen-Sutherland



Note-se que o ponto **inicial** é considerado no canto **inferior esquerdo**

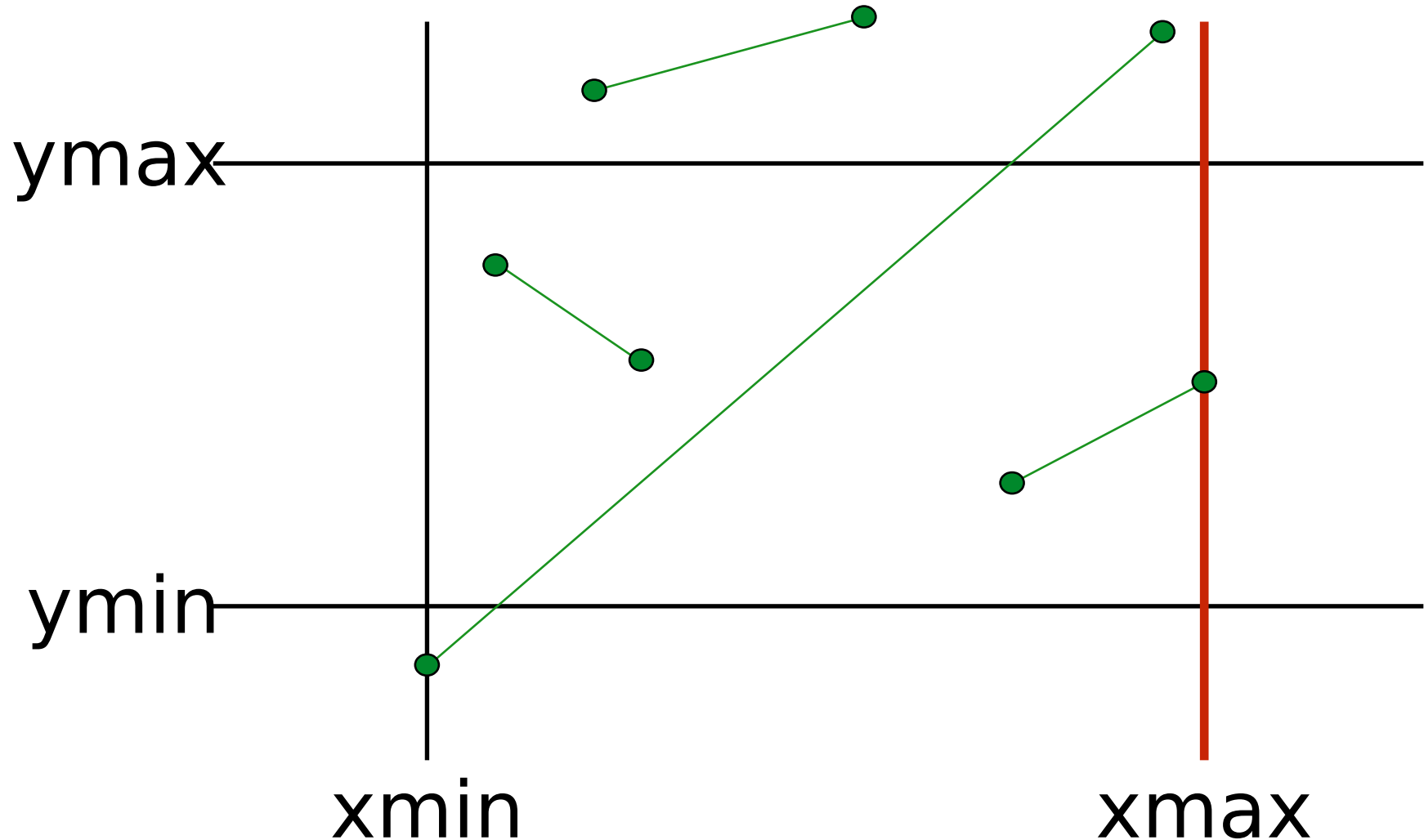
Cohen-Sutherland



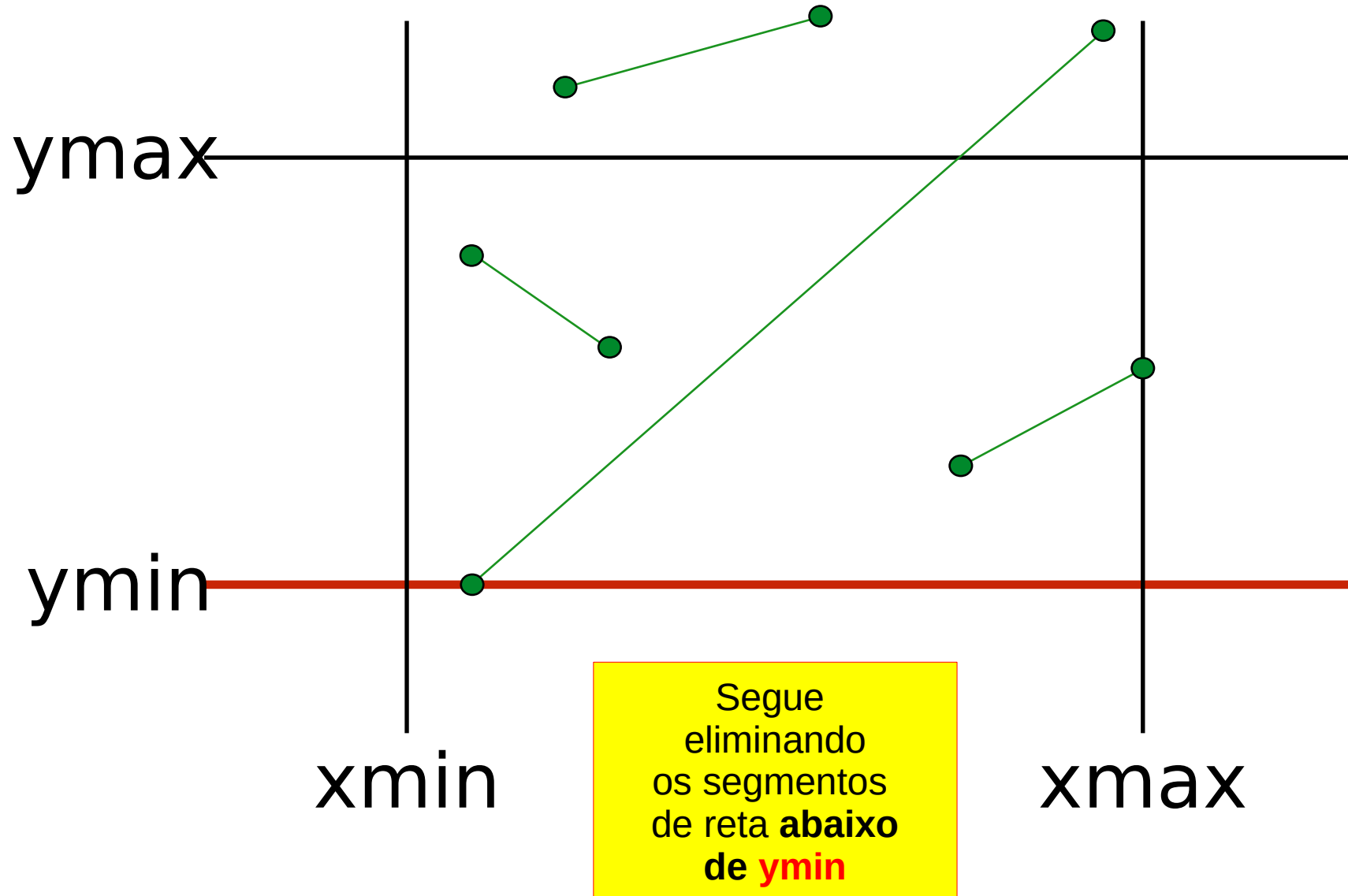
O algoritmo
começa eliminando
os segmentos
de reta à esquerda
de **xmin**

Cohen-Sutherland

Depois
elimina
os segmentos
de reta à direita
de **xmax**

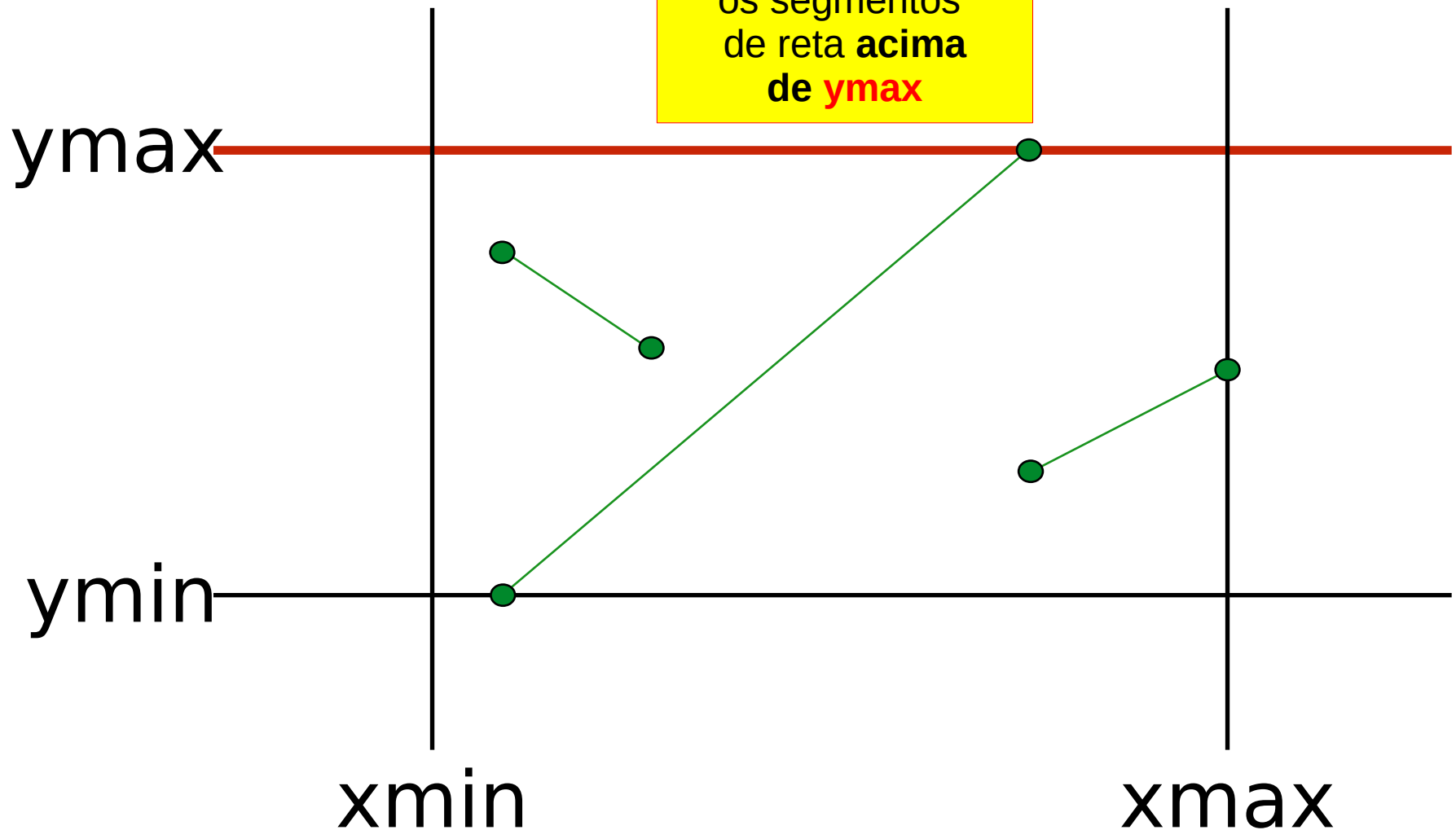


Cohen-Sutherland



Cohen-Sutherland

E termina
eliminando
os segmentos
de reta **acima**
de **y_{max}**



Cohen-Sutherland

- O recorte só é necessário se houver um vértice **dentro e outro fora** em relação a um semi-plano.
- A classificação de cada vértice pode ser codificada usando **4 bits**, um para cada semi-espço.
 - **Dentro = 0** e **Fora = 1**

Cohen-Sutherland

- O código dos pontos de cada uma das 9 áreas é formado por um número de 4 bits, da seguinte maneira:

1º bit : em **1** se o ponto está à esquerda da janela
em **0** se o ponto não está à esquerda da janela

2º bit : em **1** se o ponto está à direita da janela
em **0** se o ponto não está à direita da janela

3º bit : em **1** se o ponto está abaixo da janela
em **0** se o ponto não está abaixo da janela

4º bit : em **1** se o ponto está acima da janela
em **0** se o ponto não está acima da janela

Cohen-Sutherland

1001	1000	1010	$y = y_{\max}$
0001	0000	0010	
0101	0100	0110	$y = y_{\min}$
$x = x_{\min}$		$x = x_{\max}$	

Rejeição trivial: Classifica(P1) $\neq 0$ **and** Classifica(P2) $\neq 0$

Aceitação trivial: Classifica(P1) $== 0$ **or** Classifica(P2) $== 0$

Cohen-Sutherland

```
void CohenSutherlandLineClip(double x0, double y0, double x1, double y1,
                             double xmin, double xmax, double ymin, double ymax, Uint32 color)
{
    unsigned char outcode0, outcode1, outcodeOut;
    double x, y; bool accept = FALSE, done = FALSE;

    outcode0 = code(x0, y0, xmin, xmax, ymin, ymax);
    outcode1 = code(x1, y1, xmin, xmax, ymin, ymax);

    do {
        if ((outcode0 | outcode1) == 0) {
            accept = TRUE; done = TRUE; /* trivial draw and exit */
        } else if ((outcode0 & outcode1) != 0) {
            done = TRUE; /* trivial reject and exit */
        } else {
            outcodeOut = (outcode0 != 0) ? outcode0 : outcode1; /* discard an out part */
            if (outcodeOut & 8) { /* pick an out vertex */
                x = x0 + (x1 - x0) * (ymax - y0) / (y1 - y0); y = ymax; /* discard top */
            } else if (outcodeOut & 4) {
                x = x0 + (x1 - x0) * (ymin - y0) / (y1 - y0); y = ymin; /* discard bottom */
            } else if (outcodeOut & 2) {
                y = y0 + (y1 - y0) * (xmax - x0) / (x1 - x0); x = xmax; /* discard right */
            } else if (outcodeOut & 1) {
                y = y0 + (y1 - y0) * (xmin - x0) / (x1 - x0); x = xmin; /* discard left */
            }

            if (outcodeOut == outcode0) {
                x0 = x; y0 = y; outcode0 = code(x0, y0, xmin, xmax, ymin, ymax);
            } else {
                x1 = x; y1 = y; outcode1 = code(x1, y1, xmin, xmax, ymin, ymax);
            }
        }
    } while (!done);
    if (accept) bresenham(x0, y0, x1, y1, color);
}
```

Cohen-Sutherland

```
unsigned char code(double x, double y,  
                  double xmin, double xmax, double ymin, double ymax)  
{  
    unsigned char code=0;  
  
    if (y > ymax) code += 8;  
    if (y < ymin) code += 4;  
    if (x > xmax) code += 2;  
    if (x < xmin) code += 1;  
  
    return code;  
}
```

Cohen-Sutherland

- O algoritmo de **Cohen-Sutherland** calcula **interseções desnecessárias**, pois resolve o mesmo segmento várias vezes.
- O algoritmo de **Liang-Barsky** utiliza uma estratégia que recorta com **menor número de interseções**.

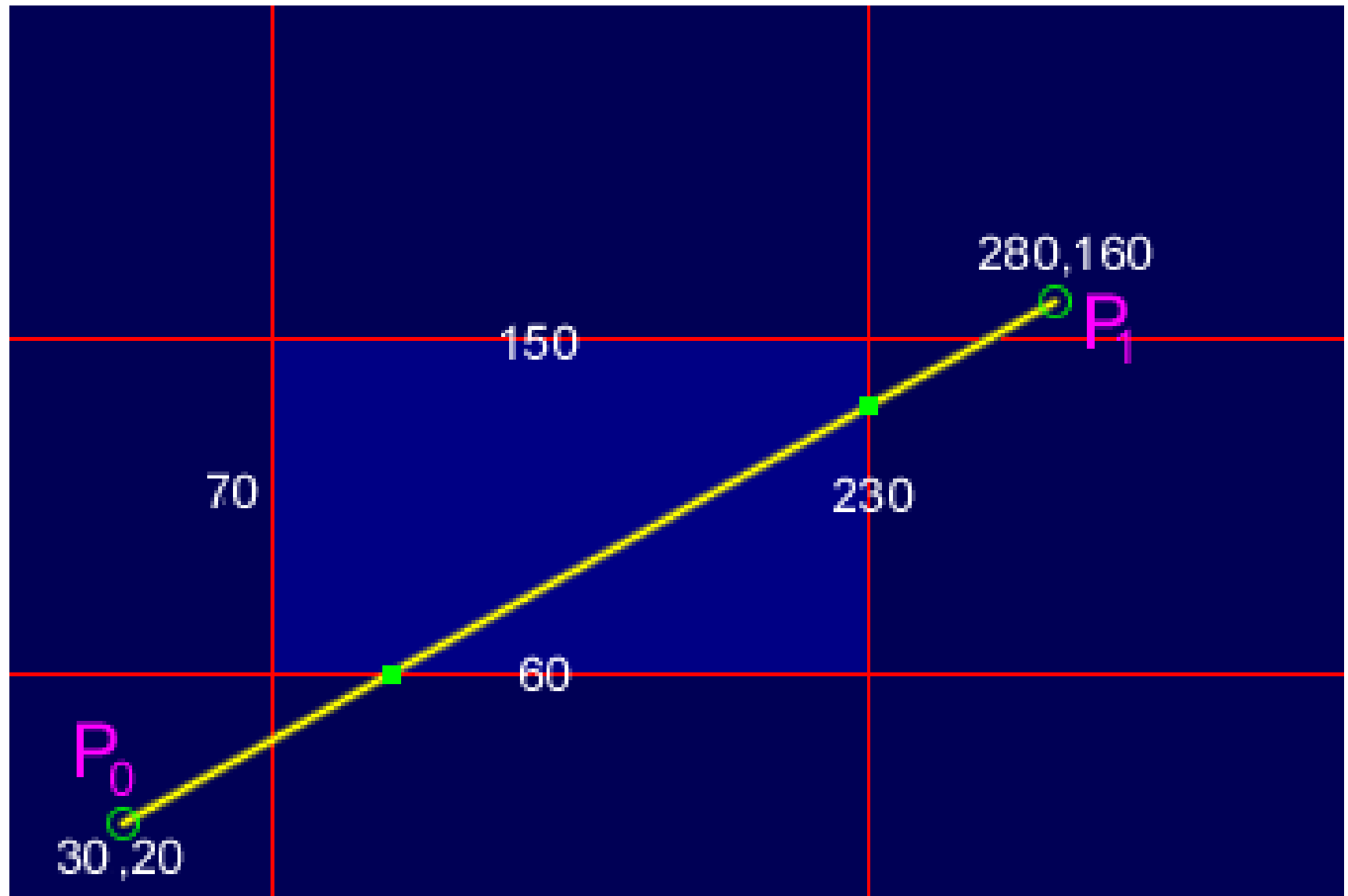
Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Liang-Barsky

- O algoritmo de **Liang-Barsky** é outro algoritmo de recorte de linha.
- É mais eficiente do que o algoritmo Cohen-Sutherland e pode ser estendido para recorte tridimensional.
- É considerado o algoritmo de recorte de linha paramétrico **mais rápido**.

Liang-Barsky



Liang-Barsky

- Os seguintes conceitos são usados neste recorte:
 - A **equação paramétrica** da linha.
 - As **desigualdades** que descrevem o intervalo da janela de recorte que é usado para **determinar as intersecções entre a linha e a janela de recorte**.

Liang-Barsky

- A equação paramétrica da reta é dada por :

$$X = x1 + t(x2-x1)$$

$$Y = y1 + t(y2-y1)$$

- Onde t assume valores entre 0 e 1

Liang-Barsky

- Portanto, descrevendo as condições de recorte na forma paramétrica:

$$x_{wmin} \leq x1 + t(x2-x1) \leq x_{wmax}$$

$$y_{wmin} \leq y1 + t(y2-y1) \leq y_{wmax}$$

- As quatro inequ岸idades acima podem ser descritas como:

$$tp_k \leq q_k$$

Liang-Barsky

- Onde $k = 1, 2, 3, 4$ (correspondem às bordas esquerda, inferior, direita e superior, respectivamente)
- Os pontos p e q são definidos como:

$$p1 = -(x2-x1), \quad q1 = x1 - x_{wmin} \text{ (Borda Esquerda)}$$

$$p2 = (x2-x1), \quad q2 = x_{wmax} - x1 \text{ (Borda Direita)}$$

$$p3 = -(y2-y1), \quad q3 = y1 - y_{wmin} \text{ (Borda Inferior)}$$

$$p4 = (y2-y1), \quad q4 = y_{wmax} - y1 \text{ (Borda Superior)}$$

Liang-Barsky

- Quando a **linha é paralela** ao limite de uma janela de visualização, **o valor p para esse limite é zero**.
- Quando **$p_k < 0$** , à medida que t aumenta, a linha vai de fora para dentro (**entrando na janela**).
- Quando **$p_k > 0$** , a linha vai de dentro para fora (**saindo da janela**).
- Quando **$p_k = 0$ e $q_k < 0$** , a linha é invisível porque está **fora da janela** de visualização.
- Quando **$p_k = 0$ e $q_k > 0$** , a linha **está dentro** do limite da janela correspondente.

Liang-Barsky

- Os parâmetros $t1$ e $t2$ podem ser calculados para definir a parte da linha que está dentro da janela de visualização.
- Quando,
 - $p_k < 0$, $\max(0, q_k / p_k)$ é usado.
 - $p_k > 0$, $\min(1, q_k / p_k)$ é usado.
- Se $t1 > t2$, a linha está completamente fora da janela do clipe e pode ser rejeitada.
- Caso contrário, os pontos finais da linha recortada são calculados a partir dos dois valores do parâmetro t .

Liang-Barsky (Código (1/3))

```
void drawClippedLine(int xmin, int ymin, int xmax, int ymax,  
    int x1, int y1, int x2, int y2, Uint32 color) {  
  
    int xx1,xx2,yy1,yy2,dx,dy;  
    float t1,t2,p[4],q[4],temp;  
  
    dx=x2-x1;  
    dy=y2-y1;  
  
    p[0]=-dx;  
    p[1]=dx;  
    p[2]=-dy;  
    p[3]=dy;  
  
    q[0]=x1-xmin;  
    q[1]=xmax-x1;  
    q[2]=y1-ymin;  
    q[3]=ymax-y1;
```

Liang-Barsky (Código (2/3))

```
for(int i=0;i<4;i++)
{
    if(p[i]==0)
    {
        // line is parallel to one of the clipping boundary
        if(q[i]>=0) {
            if(i<2) {
                if(y1<ymin) {
                    y1=ymin;
                }
                if(y2>ymax) {
                    y2=ymax;
                }
                drawLine(x1,y1,x2,y2, color);
            }
            if(i>1) {
                if(x1<xmin) {
                    x1=xmin;
                }
                if(x2>xmax) {
                    x2=xmax;
                }
                drawLine(x1,y1,x2,y2, color);
            }
        }
    }
}
```

Liang-Barsky (Código (3/3))

```
if(dx != 0 && dy != 0) {  
    t1=0;  
    t2=1;  
  
    for(int i=0;i<4;i++)  
    {  
        temp=q[i]/p[i];  
  
        if(p[i]<0)  
        {  
            if(t1<=temp)  
                t1=temp;  
        }  
        else  
        {  
            if(t2>temp)  
                t2=temp;  
        }  
    }  
  
    if(t1<t2)  
    {  
        xx1 = x1 + t1 * p[1];  
        xx2 = x1 + t2 * p[1];  
        yy1 = y1 + t1 * p[3];  
        yy2 = y1 + t2 * p[3];  
        drawLine(xx1,yy1,xx2,yy2, color);  
    }  
}
```

Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Mathes-Drakopoulos

- **Dimitrios Matthes e Vasileios Drakopoulos** propuseram em **2019** uma nova versão do algoritmo de recorte, também baseada na equação paramétrica da reta, só que mais simples e mais rápida

Mathes-Drakopoulos

- Suponha que queremos recortar uma linha dentro de uma região retangular ou janela que está definida pelos pontos **(x_{min}, y_{max})** e **(x_{max}, y_{min})** .
- Dados dois pontos **(x_1, y_1)** e **(x_2, y_2)** da linha que queremos cortar, a inclinação **m** da linha é constante é definida pela razão:

$$m = \frac{y_2 - y_1}{x_2 - x_1}.$$

Mathes-Drakopoulos

xmin

ymin

xmax

ymin



Mathes-Drakopoulos

- Para um ponto arbitrário (x, y) na linha, a razão, aplicada na fórmula prévia, pode ser escrita como:

$$m = \frac{y - y_1}{x - x_1}.$$

Mathes-Drakopoulos

- Resolvendo para y :

$$y - y_1 = m(x - x_1) \Leftrightarrow y = y_1 + m(x - x_1).$$

- Substituindo para y na equação (2):

$$y = y_1 + \frac{y_2 - y_1}{x_2 - x_1}(x - x_1).$$

- Resolvendo para x , a equação se torna (3) :

$$x = x_1 + \frac{x_2 - x_1}{y_2 - y_1}(y - y_1).$$

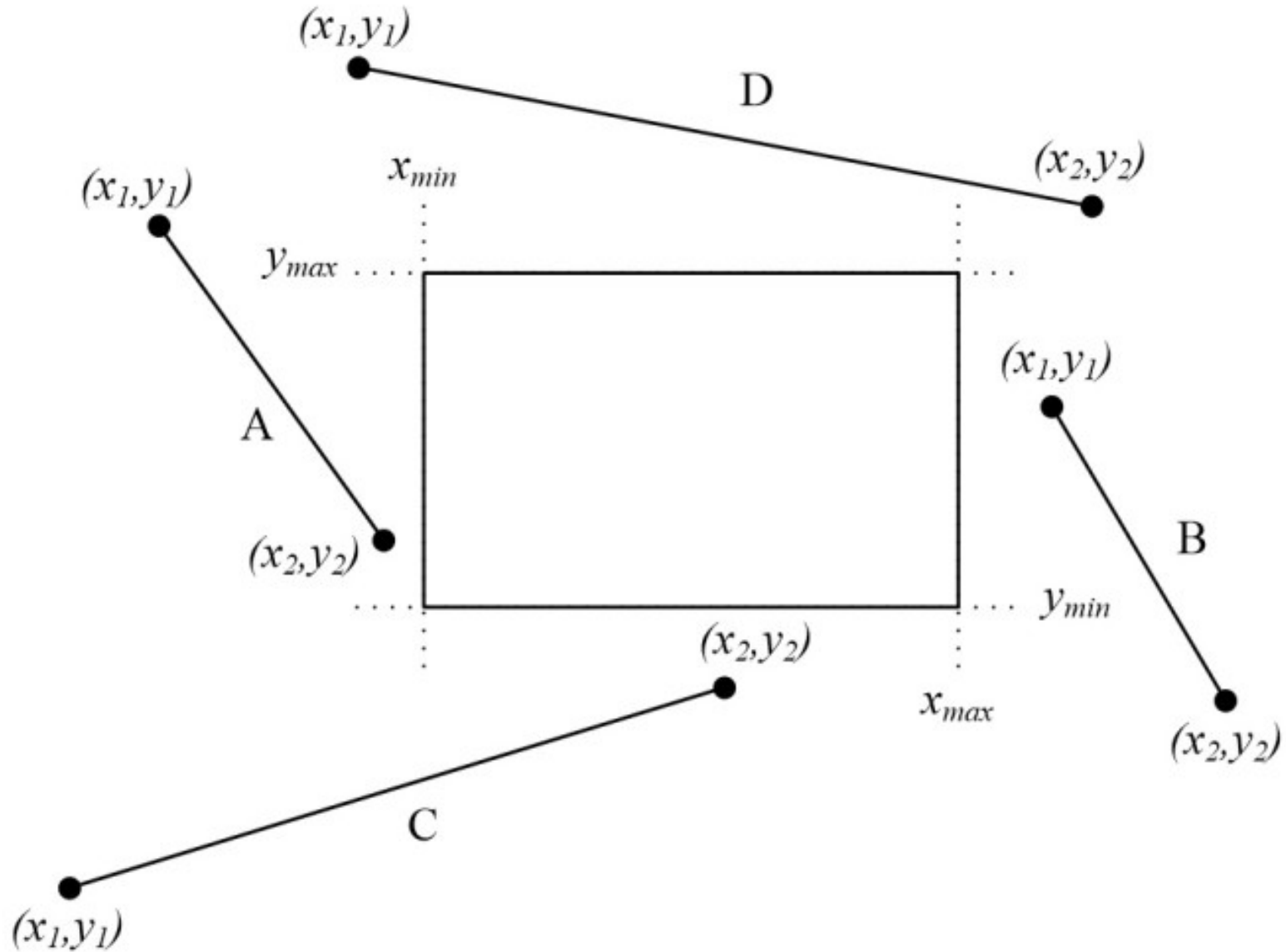
Mathes-Drakopoulos

- As equações (2) e (3) são duas representações matemáticas da equação da reta $y = mx + b$ e serão utilizadas mais tarde pelo algoritmo para determinar a parte da reta dentro da janela de recorte.

Passos do Algoritmo

- 1) As linhas estão fora da área de recorte quando seu início e fim estão totalmente aos lados, acima ou abaixo da área de recorte, ou seja:
 - Se $x1 < xmin$ e $x2 < xmin$, a linha está à esquerda da área de recorte
 - Se $x1 > xmax$ e $x2 > xmax$, a linha está à direita da área de recorte
 - Se $y1 < ymin$ e $y2 < ymin$, a linha está abaixo da área de recorte
 - Se $y1 > ymax$ e $y2 > ymax$, a linha está acima da área de recorte

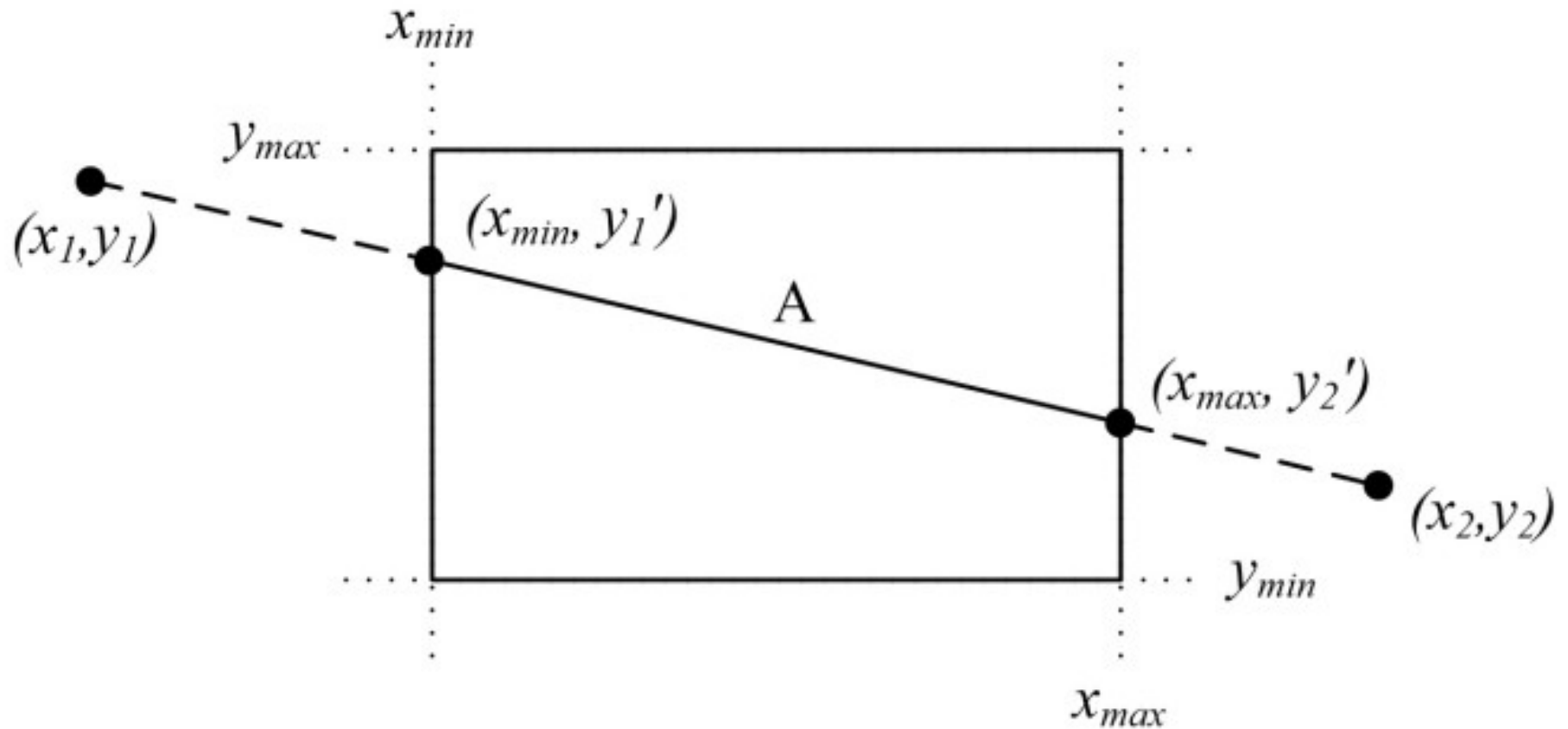
Passos do Algoritmo



Passos do Algoritmo

- 2) No segundo passo, o algoritmo compara as coordenadas dos dois pontos junto com os limites da janela de recorte.
- Ele compara cada uma das coordenadas $x1$ e $x2$ com os limites $xmin$ e $xmax$, respectivamente, bem como cada uma das coordenadas $y1$ e $y2$ com os limites $ymin$ e $ymax$.
- Caso essas coordenadas estejam fora dos limites, então o limite específico é usado na equação que determina a linha para conseguir calcular o ponto do recorte

Passos do Algoritmo



Passos do Algoritmo

- Se $x_i < x_{\min}$, então:

$$x_i = x_{\min}$$

$$y_i = y_1 + \frac{y_2 - y_1}{x_2 - x_1} (x_{\min} - x_1)$$

- Se $x_i > x_{\max}$, então:

$$x_i = x_{\max}$$

$$y_i = y_1 + \frac{y_2 - y_1}{x_2 - x_1} (x_{\max} - x_1)$$

- Onde $i = 1$ e $i = 2$

Passos do Algoritmo

- Se $y_i < y_{\min}$, então:

$$y_i = y_{\min}$$

$$x_i = x_1 + \frac{x_2 - x_1}{y_2 - y_1} (y_{\min} - x_1)$$

- Se $y_i > y_{\max}$, então:

$$y_i = y_{\max}$$

$$x_i = x_1 + \frac{x_2 - x_1}{y_2 - y_1} (y_{\max} - x_1)$$

- Onde $i = 1$ e $i = 2$

Passos do Algoritmo

- 3) Checar se os novos pontos (x_1, y_1) e (x_2, y_2) estão dentro da área de recorte.
 - Se **sim**, desenhar uma linha entre eles

Matthes-Drakopoulos : Código

```
void drawClippedLineMD(float xmin, float ymin, float xmax, float ymax,
float x1, float y1, float x2, float y2, Uint32 color){

float x[2],y[2];
int i;

if(!(x1<xmin && x2<xmin) && !(x1>xmax && x2>xmax)) {
    if(!(y1<ymin && y2<ymin) && !(y1>ymax && y2>ymax)) {

        x[0] = x1;
        y[0] = y1;
        x[1] = x2;
        y[1] = y2;

        for(i = 0; i < 2; i++){
            if(x[i]<xmin){
                x[i]=xmin;
                y[i]=((y2-y1)/(x2-x1))*(xmin-x1)+y1;
            } else if(x[i]>xmax){
                x[i]=xmax;
                y[i]=((y2-y1)/(x2-x1))*(xmax-x1)+y1;
            }
            if(y[i]<ymin){
                y[i]=ymin;
                x[i]=((x2-x1)/(y2-y1))*(ymin-y1)+x1;
            } else if(y[i]>ymax){
                y[i]=ymax;
                x[i]=((x2-x1)/(y2-y1))*(ymax-y1)+x1;
            }
        }
        if(!(x[0]<xmin && x[1]<xmin) && !(x[0]>xmax && x[1]>xmax)) {
            drawLine(x[0],y[0],x[1],y[1], color);
        }
    }
}
```

Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Polígonos

- Polígonos são objetos paramétricos compostos por uma lista circular de vértices
 - Cada vértice forma uma reta partindo de si até o próximo vértice.
 - O último vértice liga-se ao primeiro através de uma reta, fechando o polígono.

Polígonos (C++ com listas)

```
#include <list>

using namespace std;

typedef struct {int x, y;} Point;
```

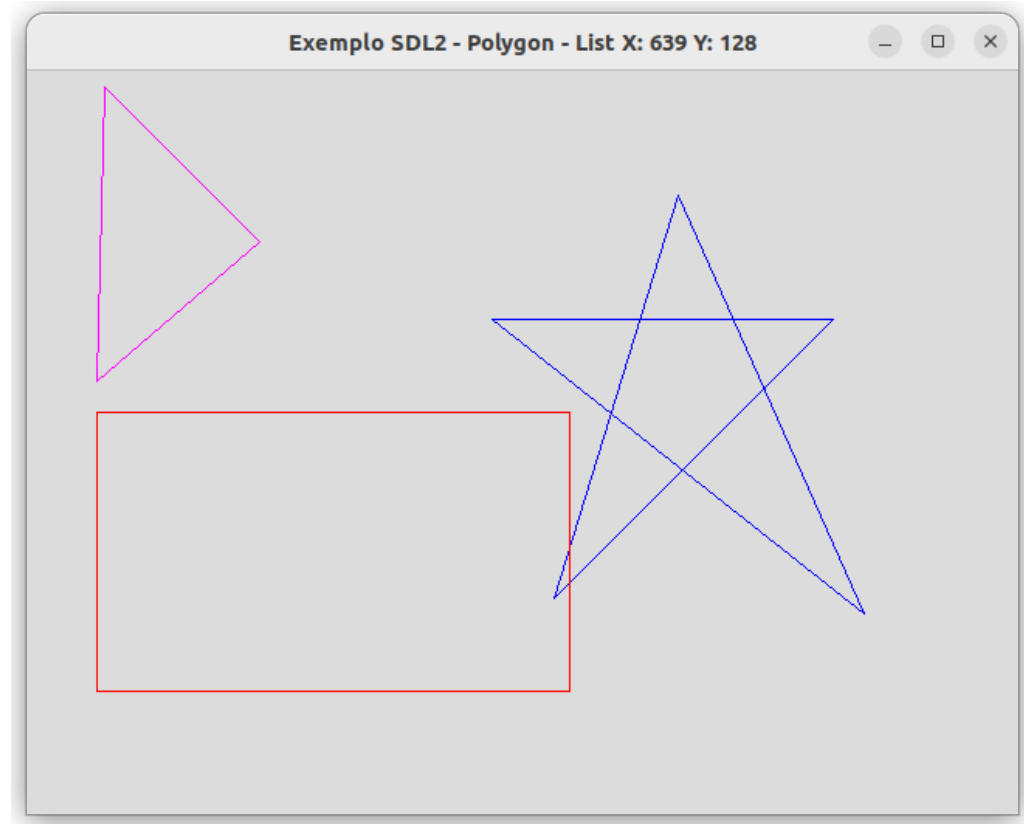
```
Point getPoint(int x, int y) {
    Point p;
    p.x = x;
    p.y = y;
    return p;
}
```

```
void drawPolygon(list<Point> pontos, Uint32 cor)
{
    Point primeiro = pontos.front();
    Point anterior = pontos.front();
    Point atual;

    int i = 0;
    for(Point p : pontos) {
        if(i>0) {
            atual = p;
            drawLine(anterior.x, anterior.y, atual.x, atual.y, cor);
            anterior = atual;
        }
        i++;
    }
    drawLine(atual.x, atual.y, primeiro.x, primeiro.y, cor);
}
```

Polígonos - Exemplo

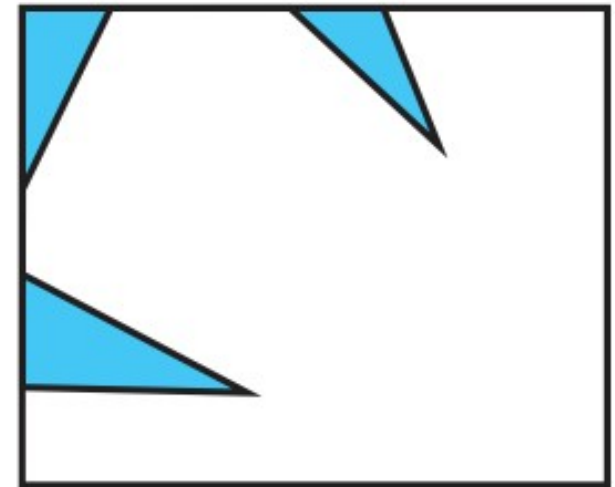
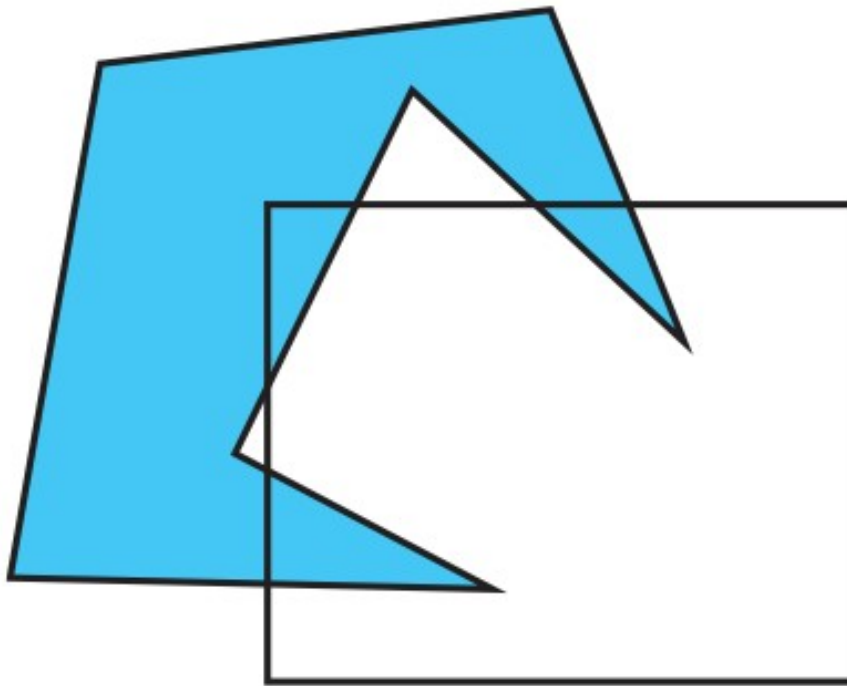
```
void display() {  
  
    list<Point> lista1;  
    list<Point> lista2;  
    list<Point> lista3;  
  
    lista1.push_back(getPoint(420,80));  
    lista1.push_back(getPoint(540, 350));  
    lista1.push_back(getPoint(300, 160));  
    lista1.push_back(getPoint(520, 160));  
    lista1.push_back(getPoint(340, 340));  
  
    lista2.push_back(getPoint(50, 10));  
    lista2.push_back(getPoint(150, 110));  
    lista2.push_back(getPoint(45, 200));  
  
    lista3.push_back(getPoint(45, 220));  
    lista3.push_back(getPoint(350, 220));  
    lista3.push_back(getPoint(350, 400));  
    lista3.push_back(getPoint(45, 400));  
  
    drawPolygon(lista1, RGB(0, 0, 255));  
    drawPolygon(lista2, RGB(255, 0, 255));  
    drawPolygon(lista3, RGB(255, 0, 0));  
}
```



Recorte de Polígonos

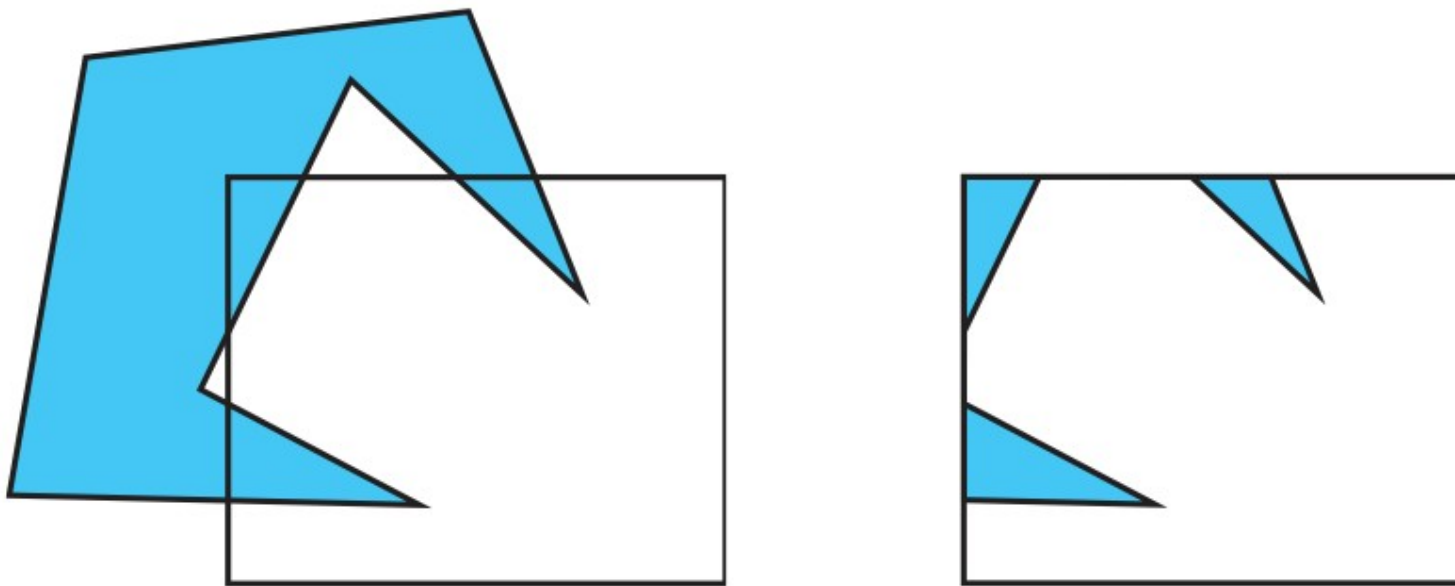
- Um algoritmo de **recorte de polígonos** pode ser obtido aplicando o algoritmo de recorte de retas **sobre cada segmento de reta que compõe** o polígono
- Polígonos podem ser representados como listas circulares de vértices
- Vetores também podem ser utilizados...

Recorte de polígonos



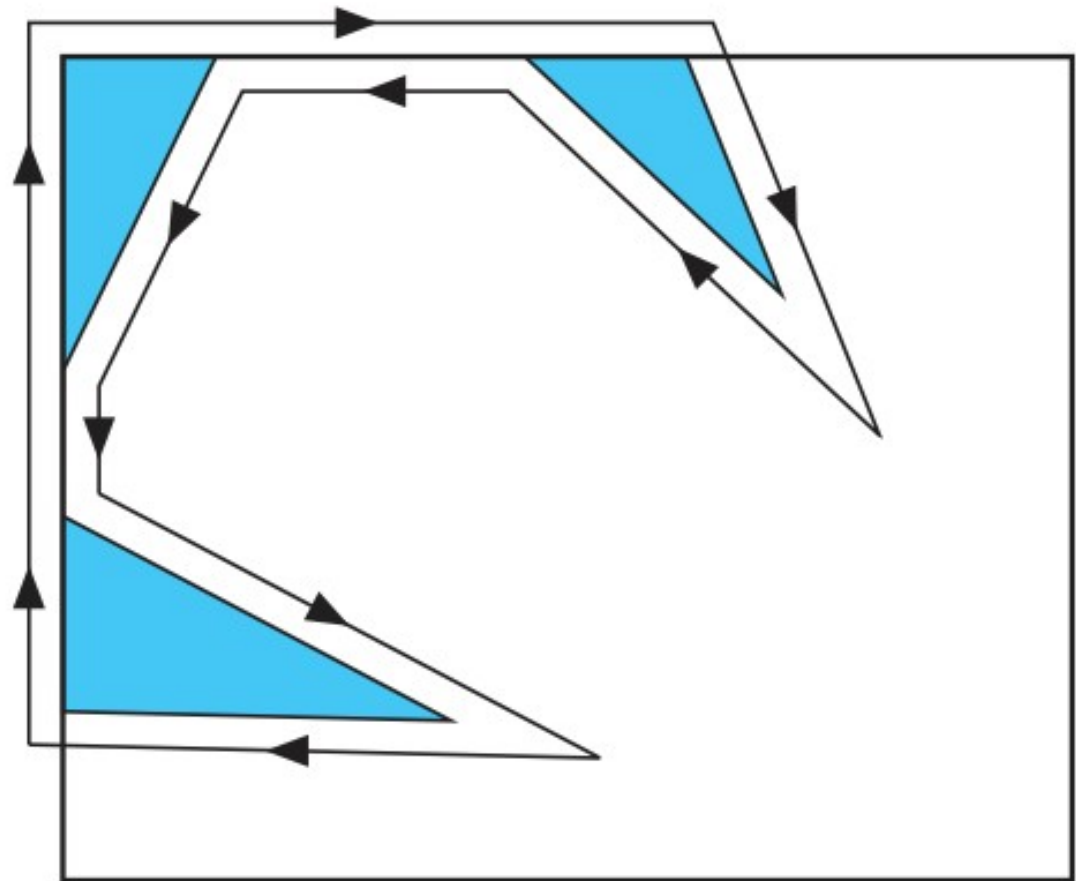
Recorte de polígonos

- Polígonos podem ser concâvos. Nesse caso, o recorte de um polígono pode gerar vários outros



Recorte de polígonos

- Uma alternativa nesse caso é conectar os componentes através de regiões degeneradas



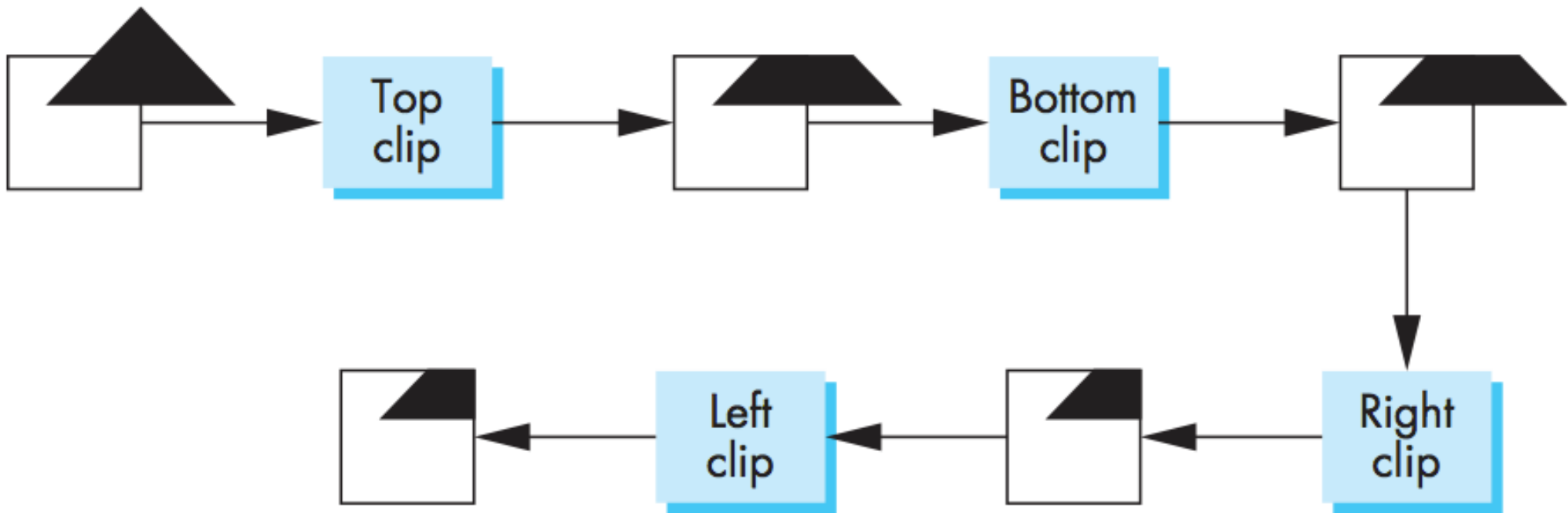
Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Algoritmo de Sutherland-Hodgman

- Um polígono é usualmente representado por uma **lista circular de vértices**.
- Os **vértices e arestas do polígono são processados em sequência** e classificados contra os semi-espacos do retângulo de recorte.

Algoritmo de Sutherland-Hodgman



Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Mapeamento

- Após os processos de **Instanciação** e de **Recorte**, as coordenadas resultantes geralmente não são compatíveis com as coordenadas da tela.
- Nesse momento é necessário realizar o **Mapeamento de coordenadas**

Coordenadas

- Lembrando-se que temos:
 - **SRO** : Sistema de Coordenadas do Objeto
 - **SRU** : Sistema de Coordenadas do Universo
 - **SRD** : Sistema de Coordenadas do Dispositivo

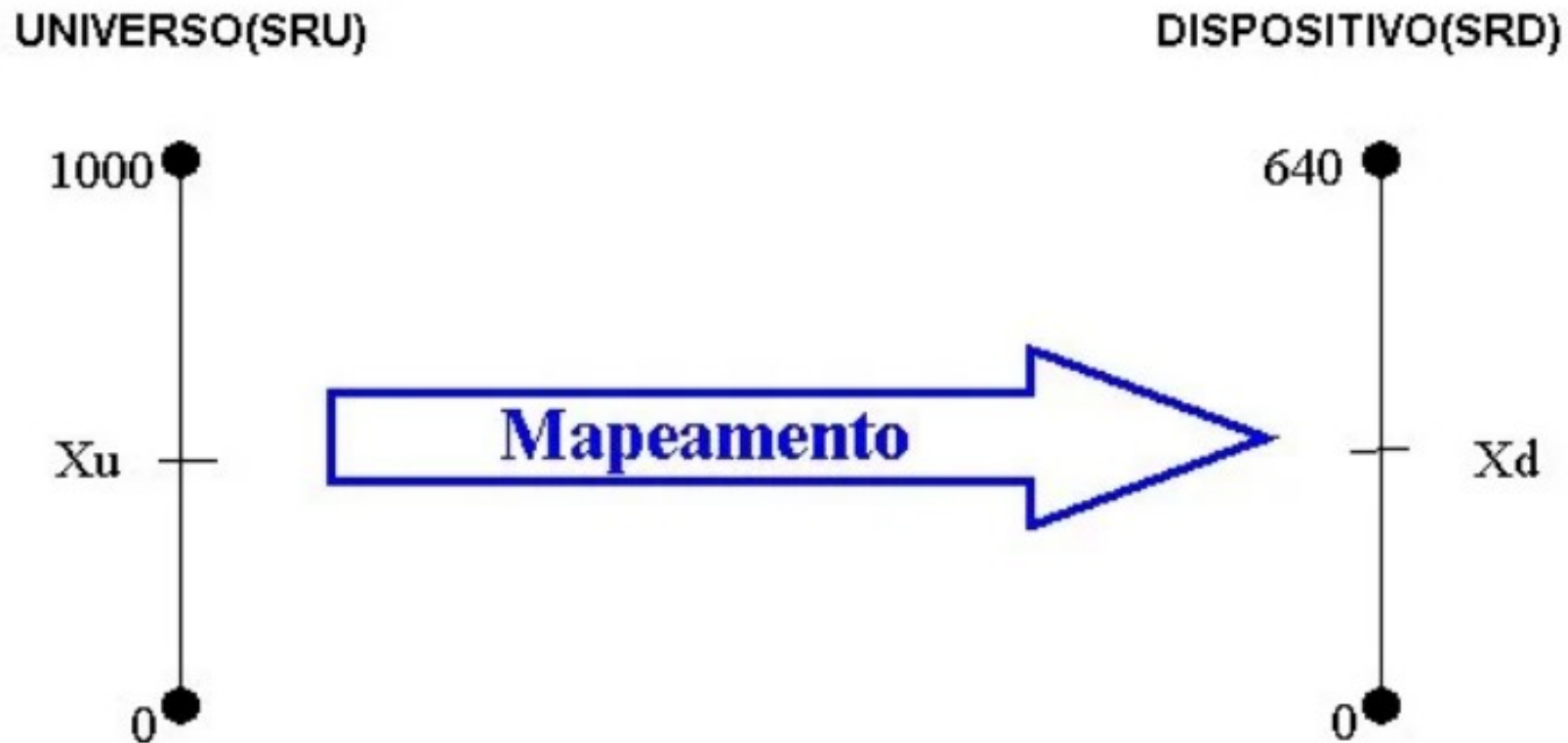
Mapeamento de coordenadas

- Para realizar o mapeamento entre estes dois Sistemas de Referência, toma-se um ponto "**P_u**" de coordenadas (**X_u**, **Y_u**), no **SRU** e tenta-se obter seu correspondente (**X_d**, **Y_d**) no **SRD**, ou seja, na tela.
- Os dados para efetuar a conversão, são os seguintes, por exemplo:

	Limites do SRU	Limites do SRD
Mínimo	(0,0)	(0,0)
Máximo	(1000,1500)	(640,480)

Mapeamento de coordenadas

- Iniciando pela componente **X** temos, de acordo com o diagrama abaixo,



Mapeamento de coordenadas

- De onde pode-se deduzir a relação:

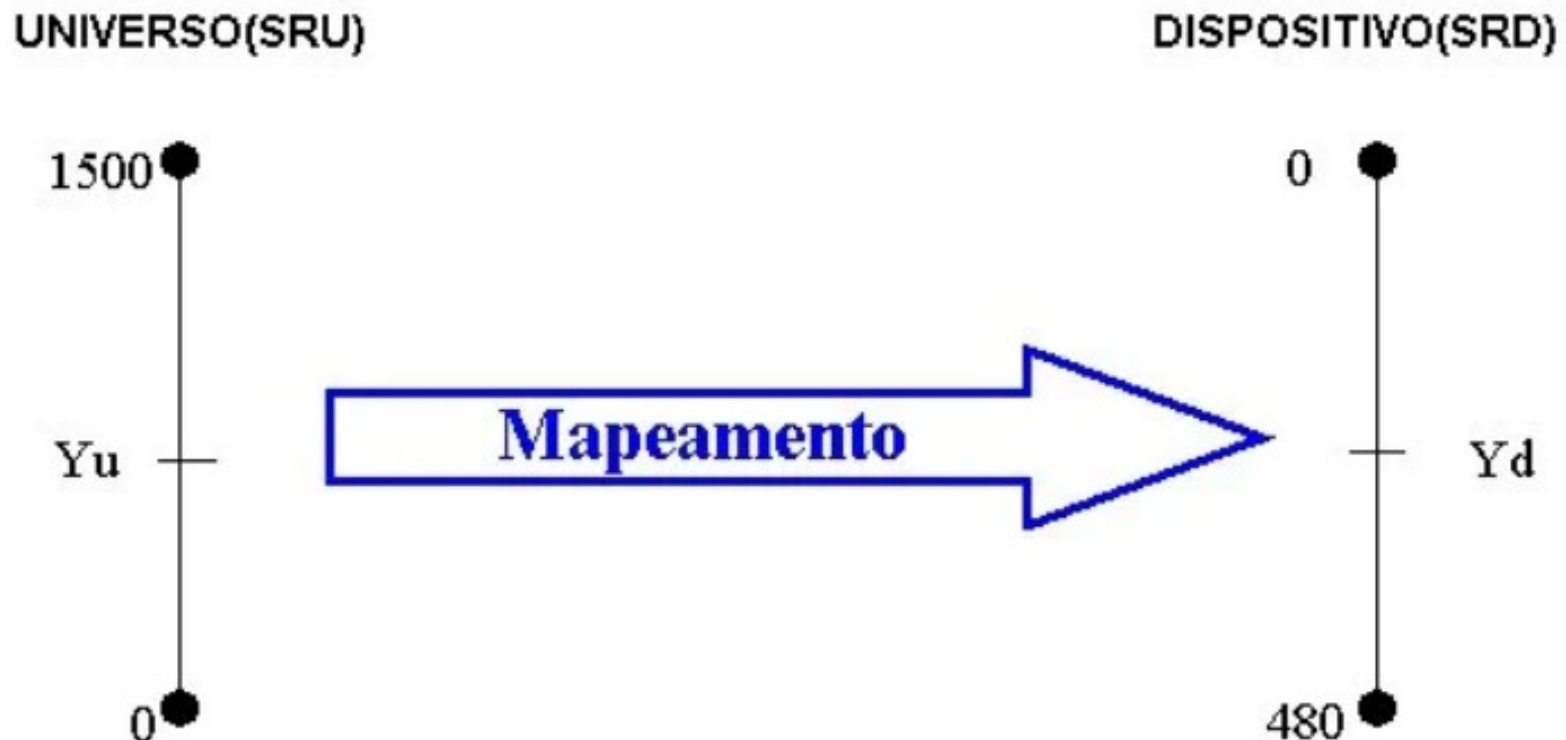
$$\frac{X_d - 0}{X_u - 0} = \frac{640 - 0}{1000 - 0}$$

- Da qual se pode concluir que:

$$X_d = \frac{X_u * 640}{1000}$$

Mapeamento de coordenadas

- Para a componente **Y** o processo é análogo, alterando-se apenas os limites dos sistemas de referência, **porém, deve-se observar que o eixo “Y” cresce em sentidos opostos nos dois sistemas:**



Mapeamento de coordenadas

- Portanto:

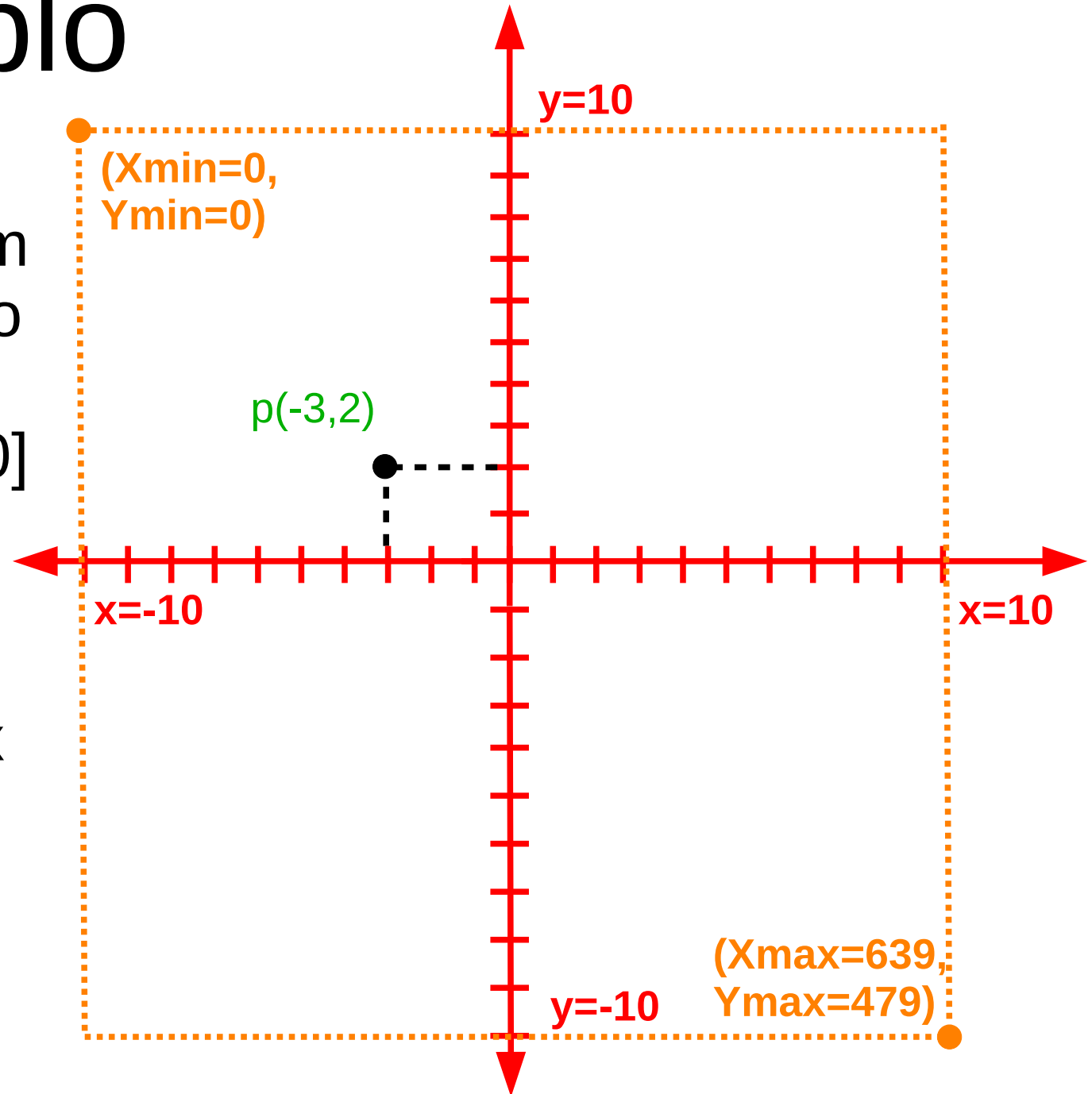
$$\frac{Y_d - 480}{Y_u - 0} = \frac{0 - 480}{1500 - 0}$$

- Da qual se pode concluir que:

$$Y_d = \frac{Y_u * (-480)}{1500} + 480$$

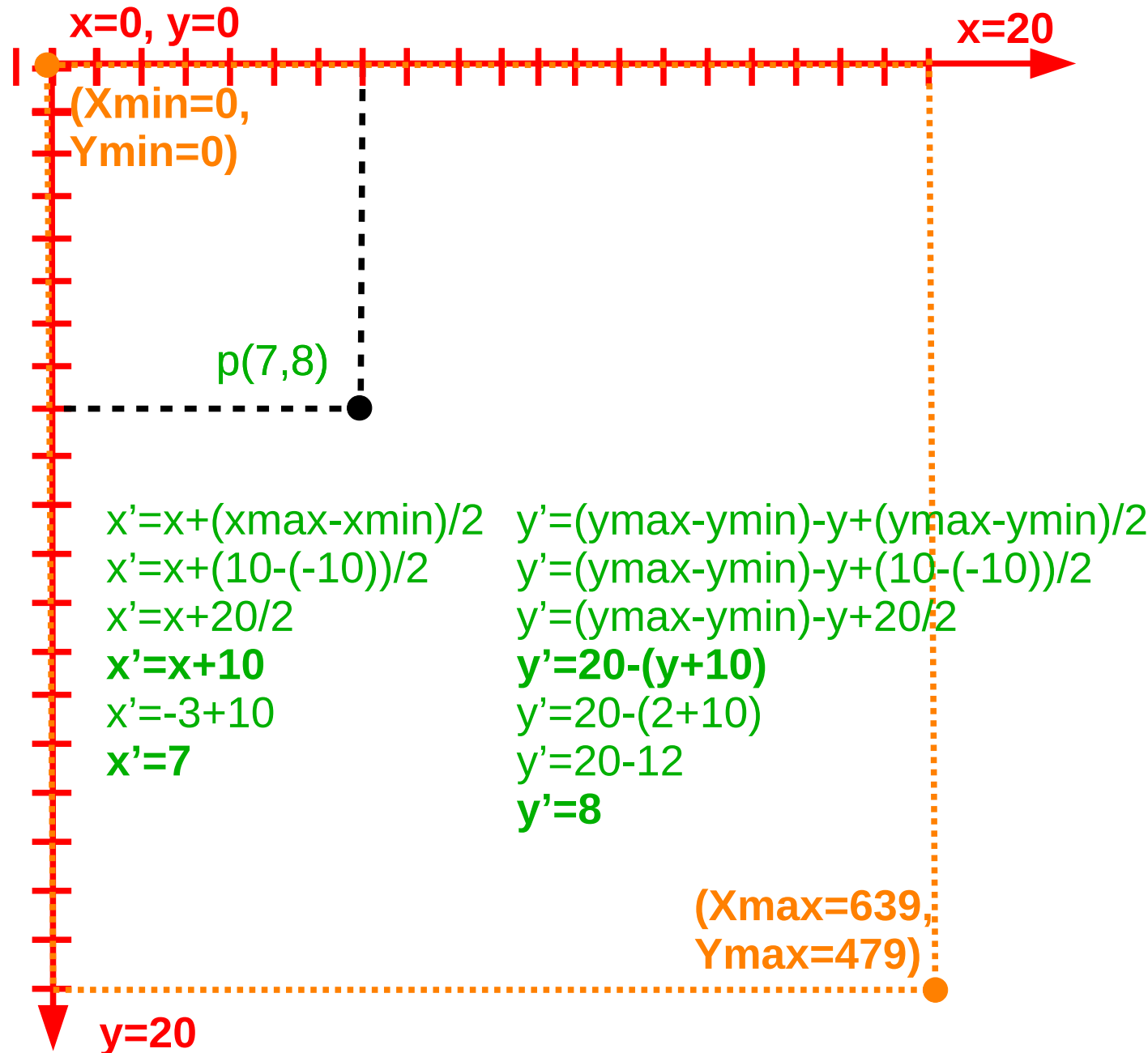
Exemplo

- Para mapear um plano cartesiano com x no intervalo $[-10,10]$ e y no intervalo $[-10,10]$ para uma tela com resolução 640 x 480 pixels



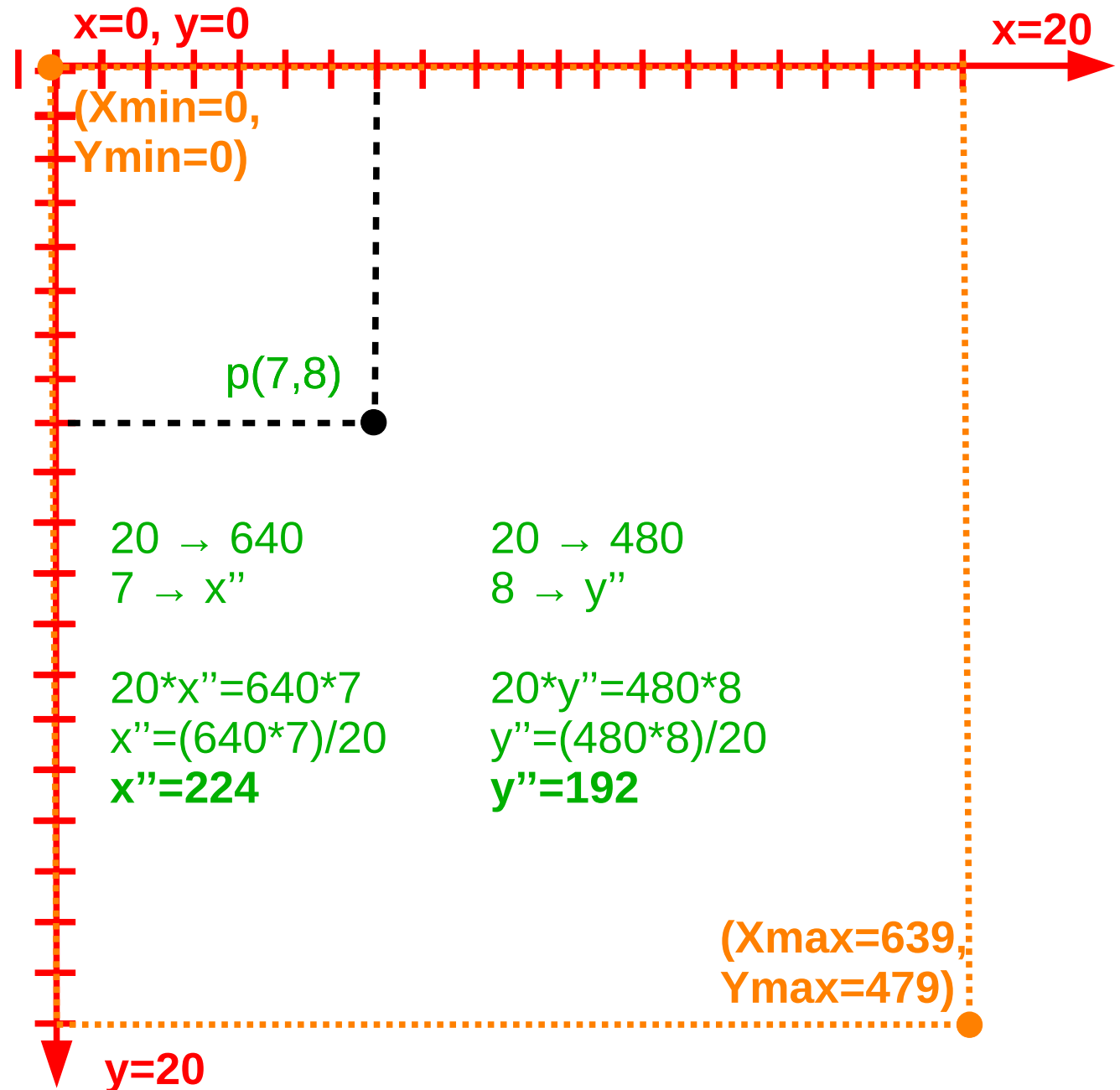
Exemplo

- Primeiro vamos deslocar os eixos x e y, acrescentando **10** unidades a cada um deles, para nos livrarmos dos números negativos. Isso desloca o ponto central para o ponto de início da janela
- Essas **10** unidades representam metade da altura e da largura



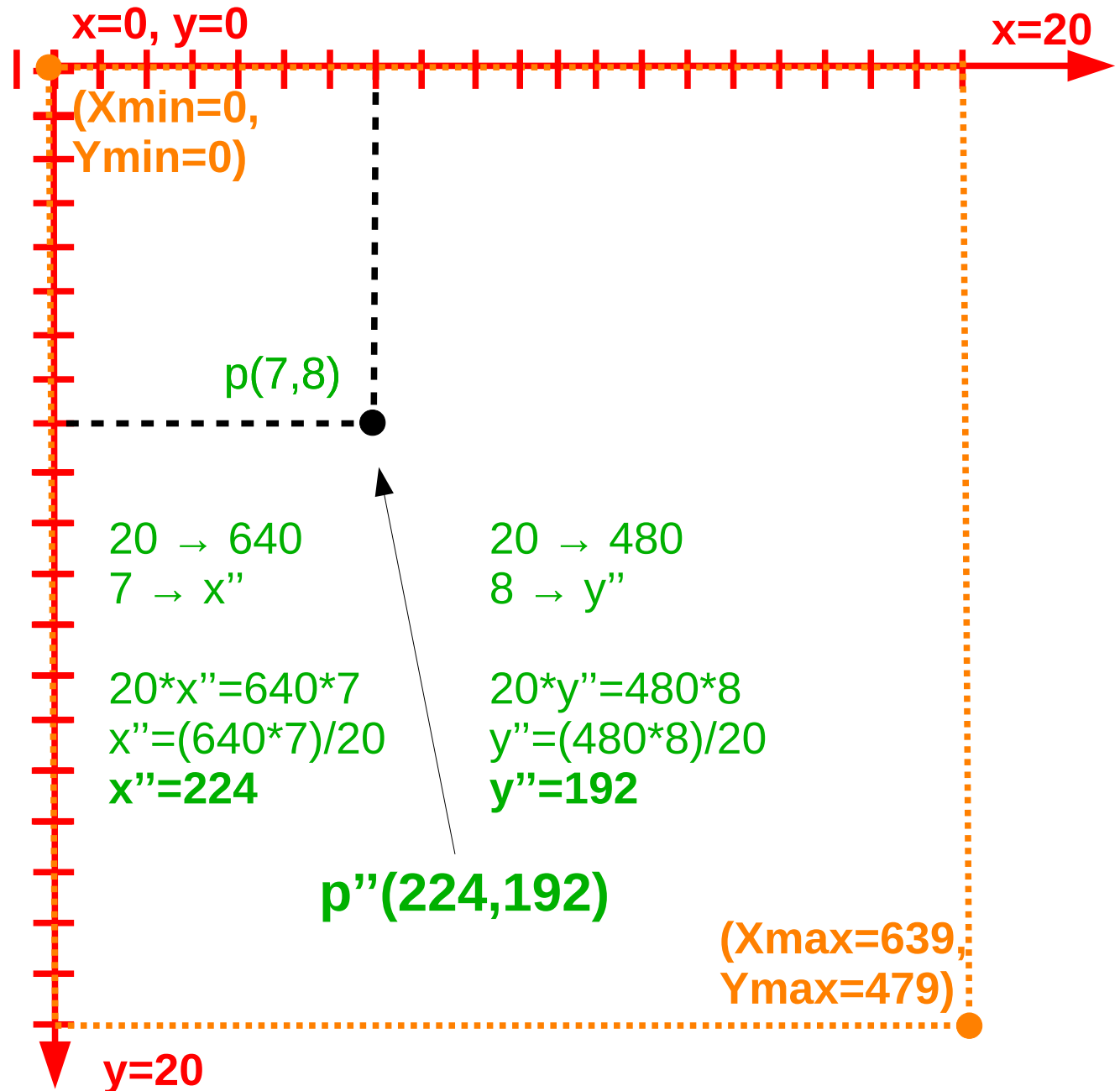
Exemplo

- Depois, usa-se a clássica regra de três!



Exemplo

- Depois, usa-se a clássica regra de três!



Roteiro

- Processo de Visualização 2d (Bidimensional)
 - Modelos x Imagens
 - Etapas do processo de Visualização 2D
 - Instanciação
 - Recorte
 - Retas
 - Cohen-Sutherland
 - Liang-Barsky
 - Matthes-Drakopoulos
 - Polígonos
 - Sutherland-Hodgman
 - Mapeamento
 - Janela de Seleção X Janela de Exibição

Janela de Seleção X Janela de Exibição

- Quando definimos que a área de trabalho da aplicação vai de (0,0) a (1000,1500), por exemplo, estamos selecionando a região do plano cartesiano com a qual desejamos trabalhar.
- A esta região dá-se o nome de **Janela de Seleção** ou **Window**.
- Dependendo da aplicação, podemos diminuir esta área, num processo conhecido como **ZOOM**.
- Esta modificação é em geral definida pelo usuário da aplicação.

Janela de Seleção X Janela de Exibição

- Quando definimos que a área de exibição, dentro da tela, vai de (0,0) a (640, 480), estamos especificando a **Janela de Exibição** ou **ViewPort**. A modificação de uma *Viewport* é, em geral, definida pelo programador da aplicação.
- Estas regiões, dependendo da aplicação, ou mesmo do momento, podem variar. A fim de tornar as fórmulas de mapeamento genéricas quanto às **Janelas de Seleção/Exibição** escolhidas, basta definir variáveis que representem as coordenadas de seus cantos

Janela de Seleção X Janela de Exibição

- Exemplificando :

```
typedef struct {  
    float x, y;  
} XYR;
```

```
XYR MIN_SEL; // canto inferior esquerdo da janela de seleção  
XYR MAX_SEL; // canto superior direito da janela de seleção  
XYR MIN_EXB; // canto inferior esquerdo da janela de exibição  
XYR MAX_EXB; // canto superior direito da janela de exibição  
|
```

Janela de Seleção X Janela de Exibição

- Utilizando a *struct* definida, as funções de mapeamento são calculadas da seguinte forma :

$$X_d = \frac{(X_u - \text{MIN_SEL.X}) * (\text{MAX_EXB.X} - \text{MIN_EXB.X})}{\text{MAX_SEL.X} - \text{MIN_SEL.X}} + \text{MIN_EXB.X}$$

$$Y_d = \frac{(Y_u - \text{MIN_SEL.Y}) * (\text{MIN_EXB.Y} - \text{MAX_EXB.Y})}{\text{MAX_SEL.Y} - \text{MIN_SEL.Y}} + \text{MAX_EXB.Y}$$

Dúvidas?



Atividade 1 :

- Crie um projeto chamado **SDL_Clipping**, e implemente nele o algoritmo de **Liang-Barsky** e/ou o algoritmo de **Matthes-Drakopoulos**
 - (Reaproveite o código do projeto **SDL_Lines** implementado na aula 02)
- Desenhe um retângulo centralizado na janela, e desenhe ao menos 6 linhas retas, que comecem, terminam ou atravessam o retângulo em diversos sentidos.
- Só devem ser exibidos na tela os segmentos de reta que estão contidos dentro do retângulo

Atividade 2 :

- No mesmo projeto, crie uma função para desenhar um polígono.
 - Essa função deverá receber por parâmetro um vetor da estrutura Point, mais um inteiro indicando o número de pontos no vetor e uma cor RGB.
 - A função deverá desenhar o polígono unindo os pontos consecutivos do vetor através de linhas retas da cor informada. O último ponto deverá ser ligado ao primeiro.
- Desenhe dois ou mais polígonos na tela, utilizando sua função

Atividade 3 :

- Faça uma nova versão da função que desenha polígonos, utilizando a lógica do algoritmo de ***Sutherland-Hodgman*** para desenharm apenas as partes do polígono que estão dentro do retângulo desenhado na Atividade 1